

Redes Neurais Convolucionais

Davi Duarte de Paula
davi.duarte@unesp.br



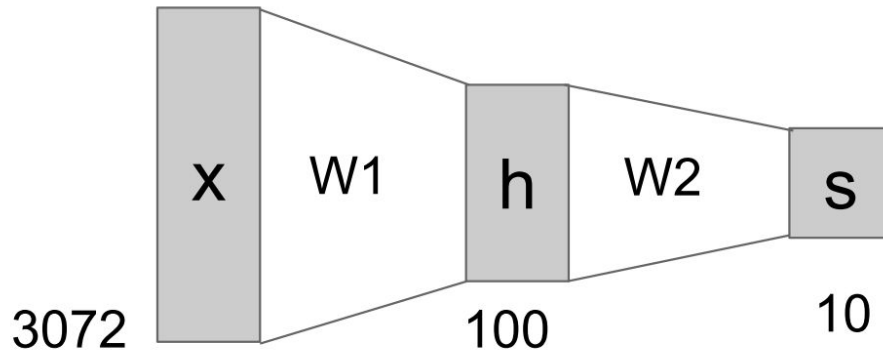
Até agora:

Linear score function:

$$f = Wx$$

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$



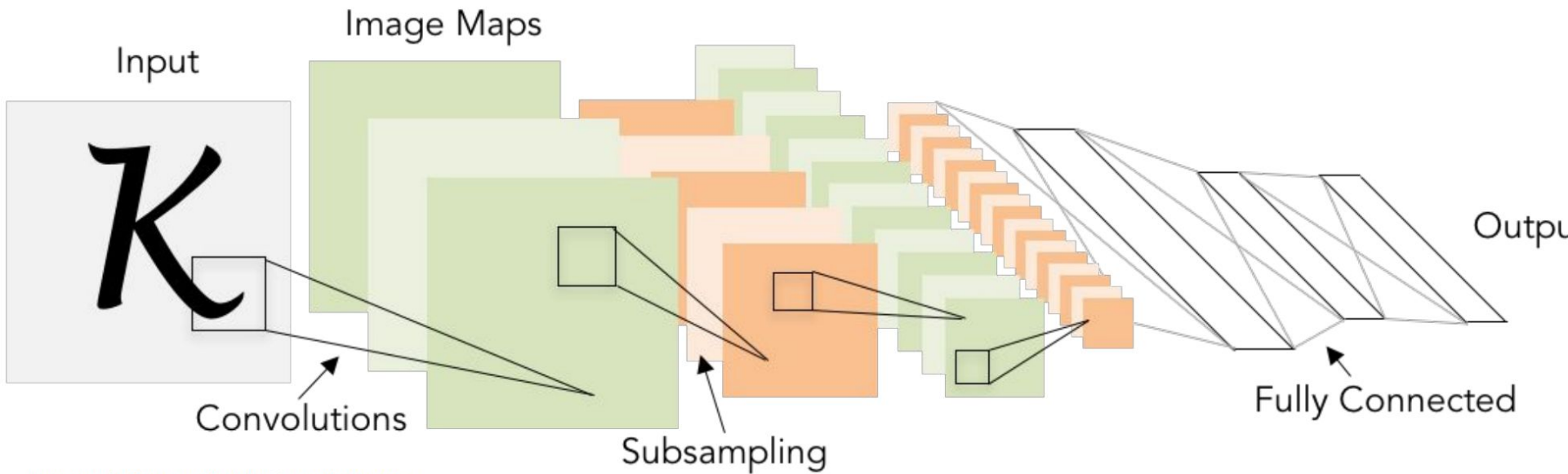


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

Um pouco de história

A máquina Perceptron Mark I foi a primeira implementação do algoritmo perceptron.

A máquina estava conectada a uma câmera que usava fotocélulas de sulfeto de cádmio 20×20 para produzir uma imagem de 400 pixels.

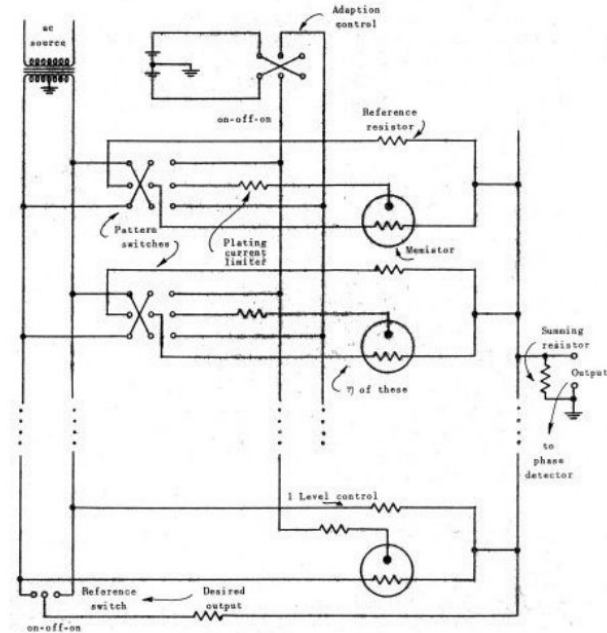
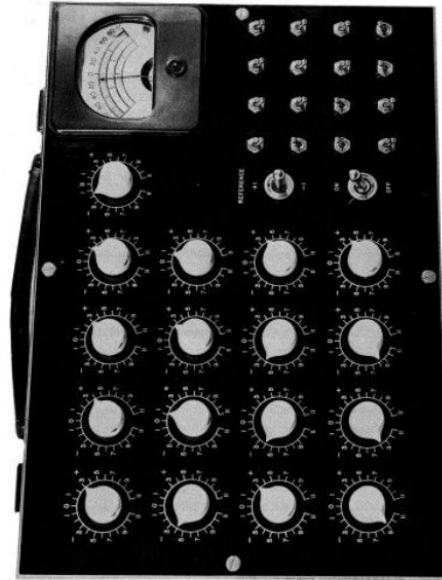
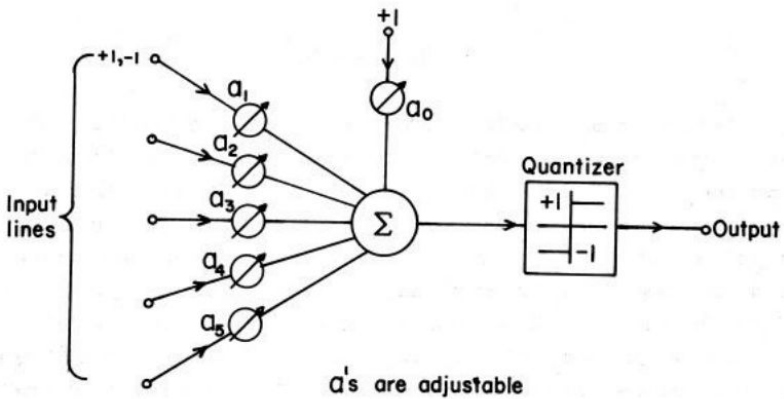
Reconhecia letras do alfabeto.

update rule:

$$w_i(t + 1) = w_i(t) + \alpha(d_j - y_j(t))x_{j,i}$$



Um pouco de história



Widrow and Hoff, ~1960: Adaline/Madaline

These figures are reproduced from [Widrow 1960, Stanford Electronics Laboratories Technical Report](#) with permission from [Stanford University Special Collections](#).

Um pouco de história

Rumelhart et al., 1986: First time back-propagation became popular

$$\frac{\partial E_p}{\partial w_{ji}} = \frac{\partial E_p}{\partial o_{pj}} \frac{\partial o_{pj}}{\partial w_{ji}}$$

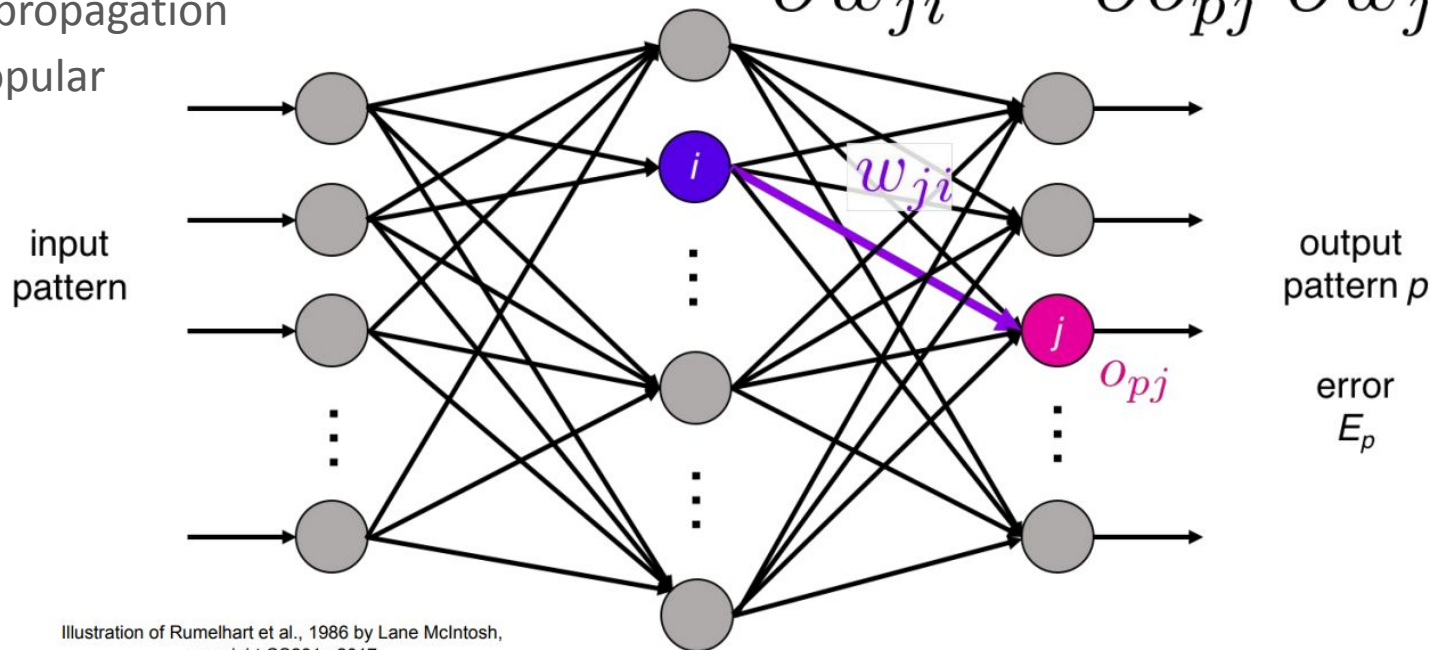
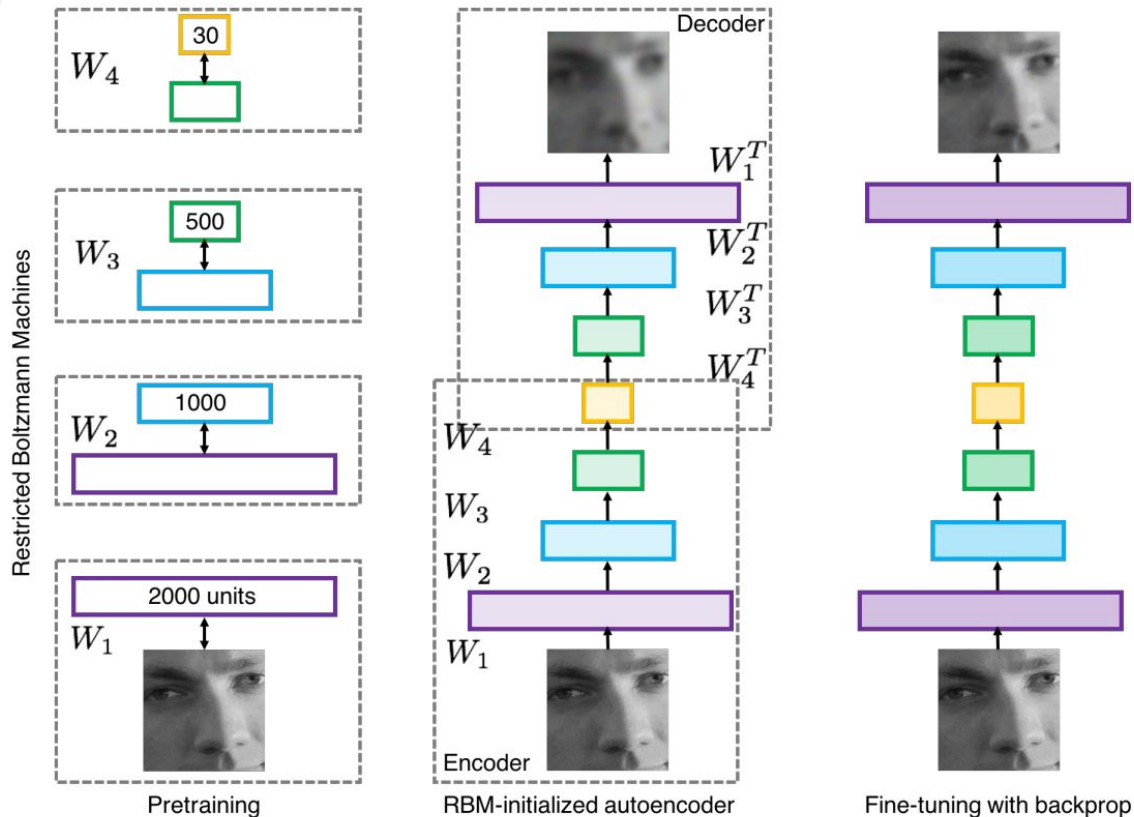


Illustration of Rumelhart et al., 1986 by Lane McIntosh,
copyright CS231n 2017

Um pouco de história

[Hinton and Salakhutdinov 2006]

Pesquisa revigorada em **Aprendizado Profundo** (Deep Learning).



Primeiro Um pouco de história e resultados fortes

Acoustic Modeling using Deep Belief Networks

Abdel-rahman Mohamed, George Dahl, Geoffrey Hinton, 2010

Context-Dependent Pre-trained Deep Neural Networks for Large Vocabulary Speech Recognition

George Dahl, Dong Yu, Li Deng, Alex Acero, 2012

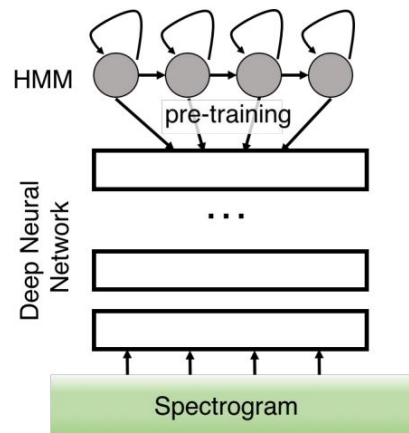
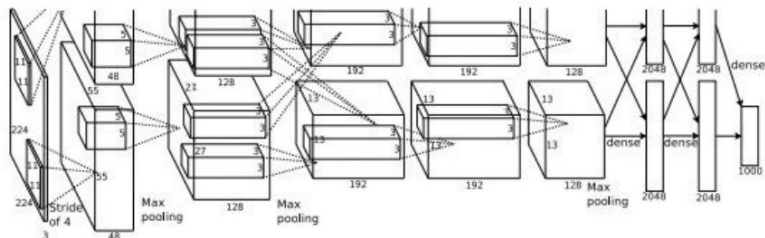


Illustration of Dahl et al. 2012 by Lane McIntosh, copyright CS231n 2017

Imagenet classification with deep convolutional neural networks

Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton, 2012

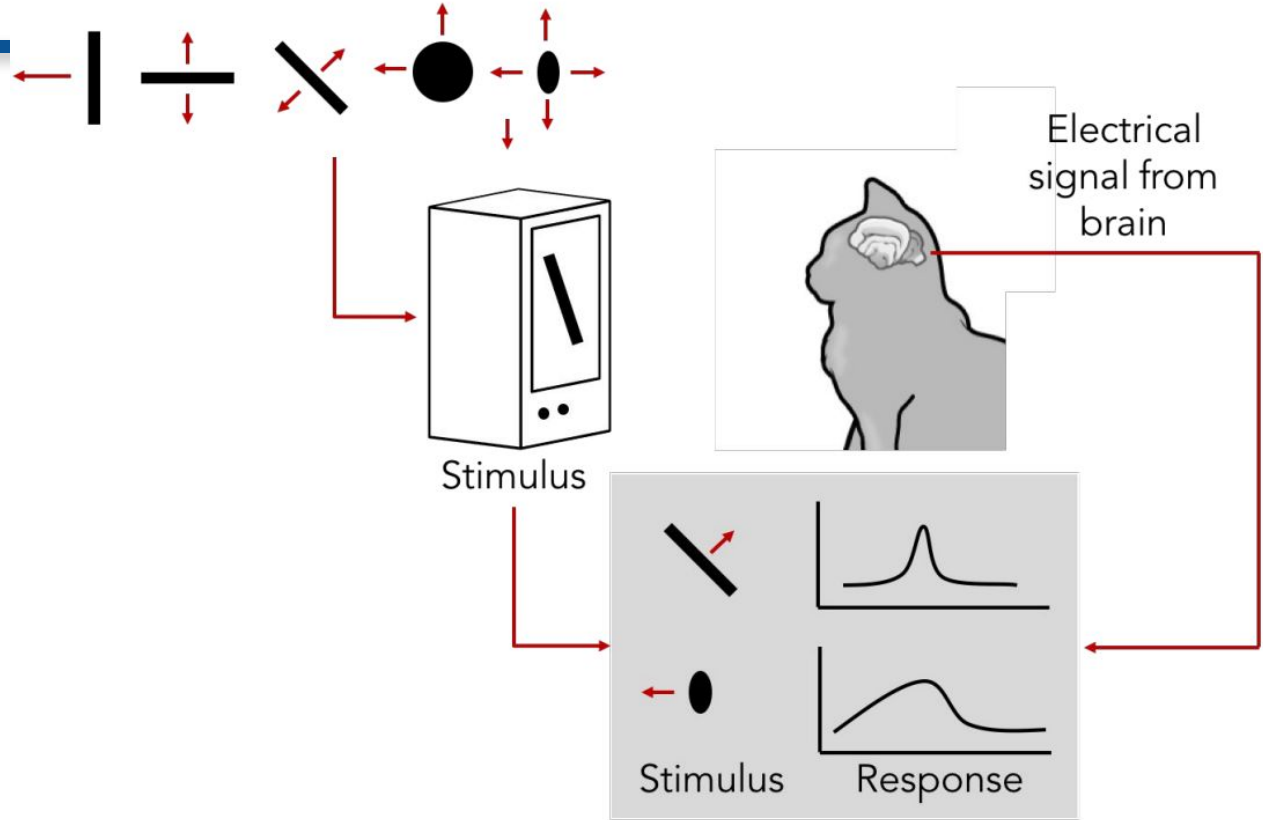


Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

Um pouco de história

Hubel & Wiesel, 1959
Campos Receptivos de Neurônios Individuais no Córtex Estriado de Gatos
(*Receptive Fields of Single Neurons in the Cat's Striate Cortex*)

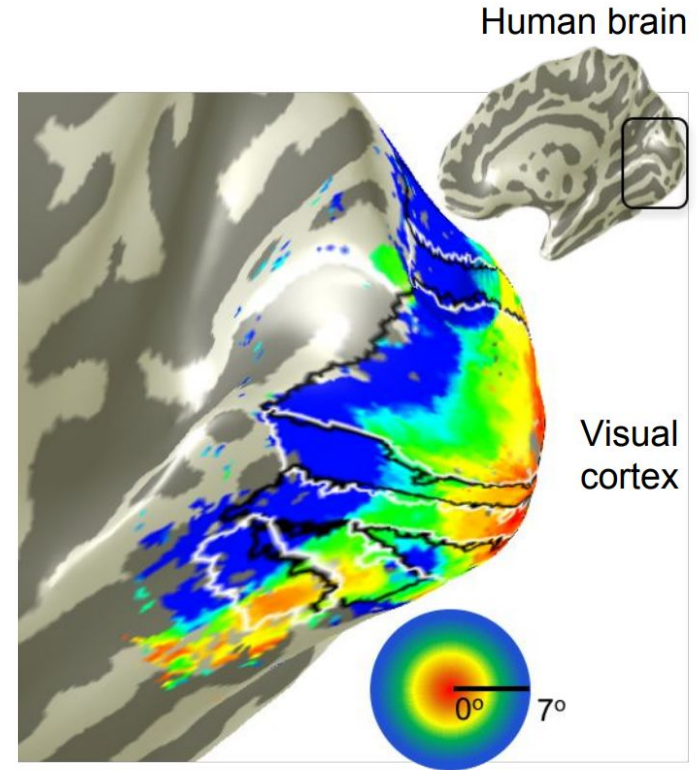
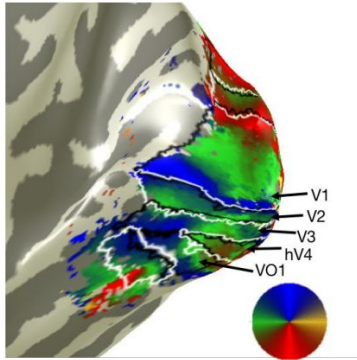
1962
Campos Receptivos, Interação Binocular e Arquitetura Funcional no Córtex Visual de Gatos
(*Receptive Fields, Binocular Interaction and Functional Architecture in the Cat's Visual Cortex*)



Um pouco de história

Mapeamento topográfico no córtex:

neurônios vizinhos no córtex representam regiões vizinhas no campo visual.



Retinotopy images courtesy of Jesse Gomez in the Stanford Vision & Perception Neuroscience Lab.

Um pouco de história

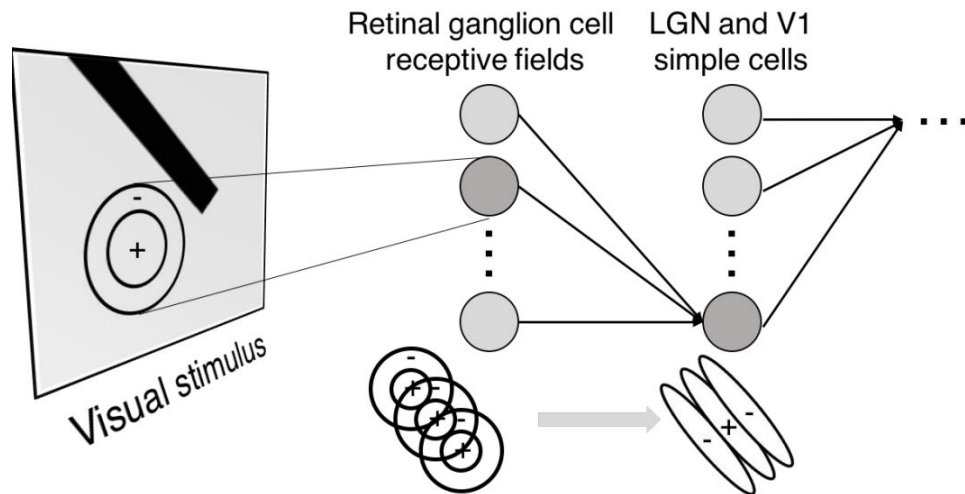
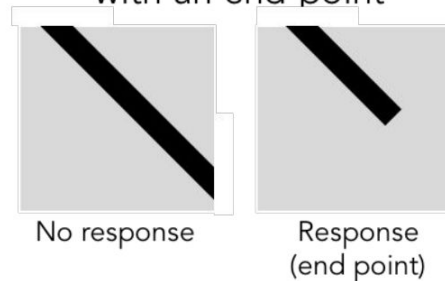


Illustration of hierarchical organization in early visual pathways by Lane McIntosh, copyright CS231n 2017

Simple cells:
Response to light
orientation

Complex cells:
Response to light
orientation and movement

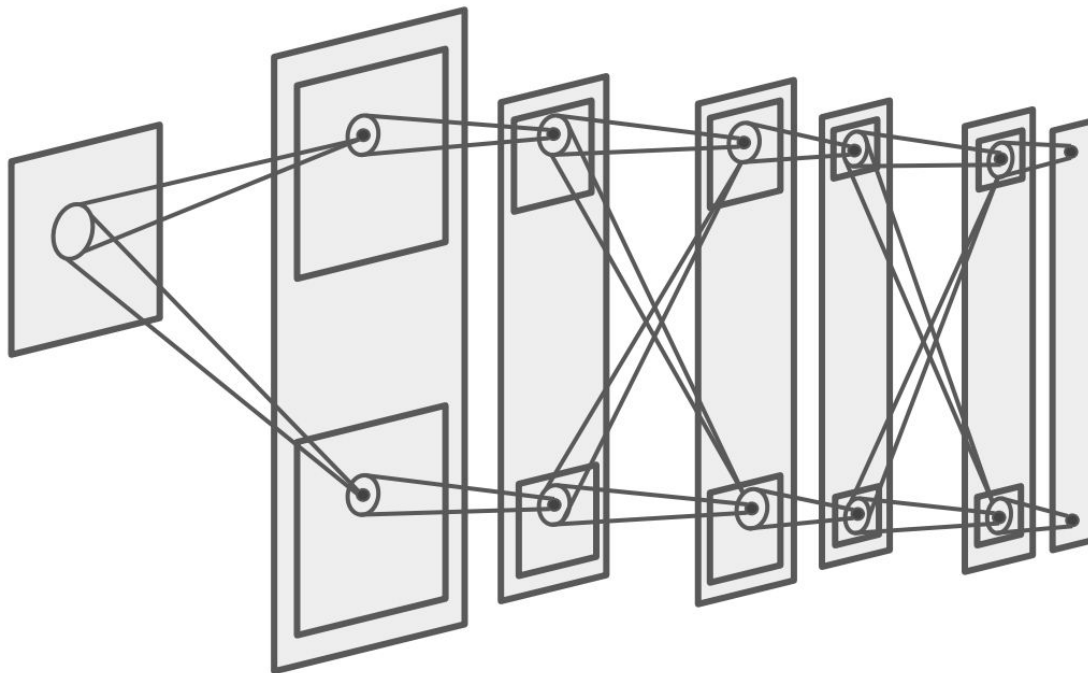
Hypercomplex cells:
response to movement
with an end point



Neocognitron *[Fukushima 1980]*

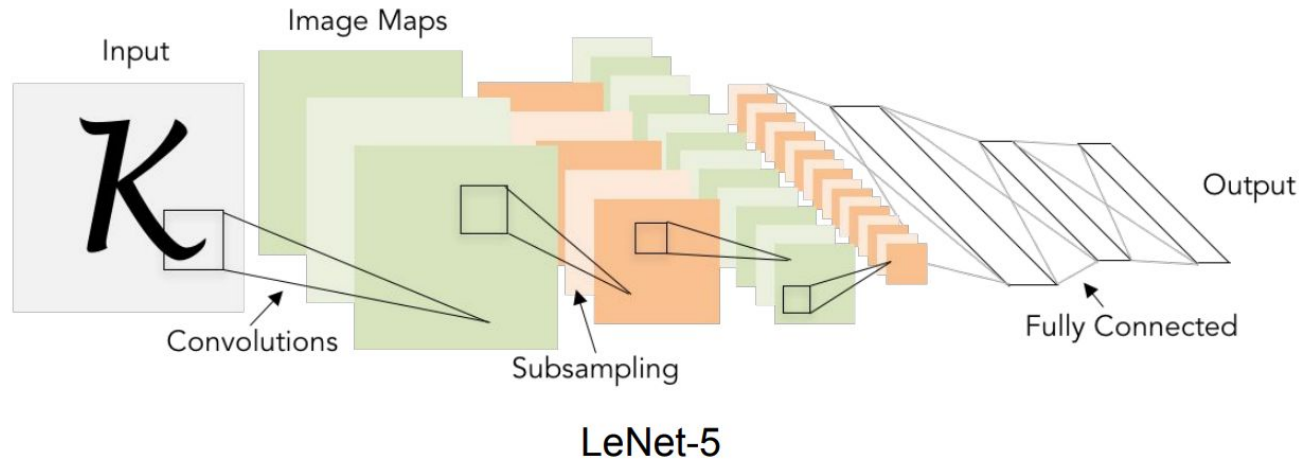
Arquitetura em “sanduíche” (SCSCSC...)

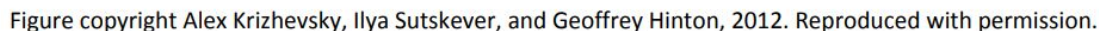
- **Células simples:** possuem **parâmetros modificáveis** (detectam padrões específicos como arestas, orientações, etc.)
- **Células complexas:** realizam **pooling** (integram as respostas das células simples, conferindo invariância a posição e escala)



Gradient-based learning applied to document recognition

[LeCun, Bottou, Bengio, Haffner 1998]

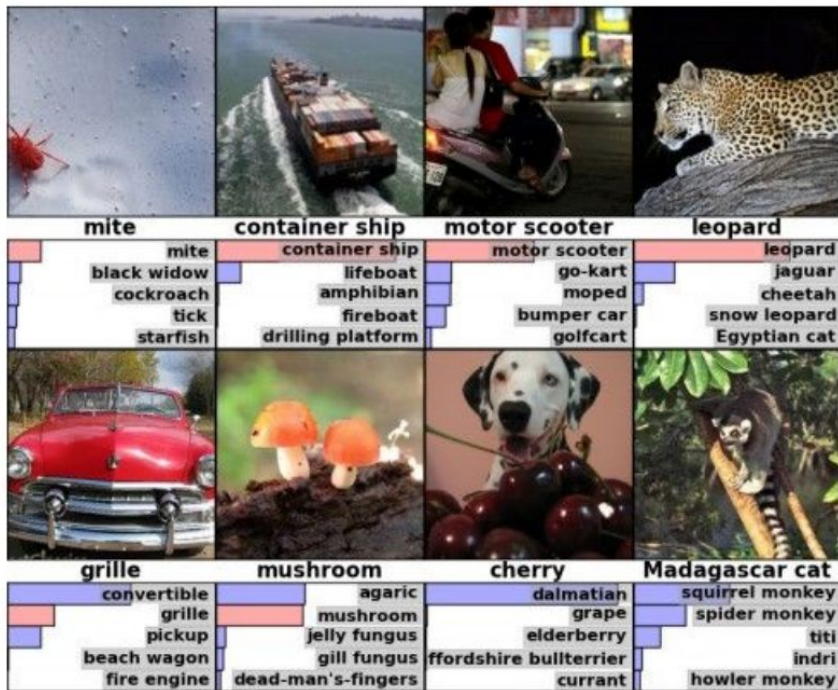




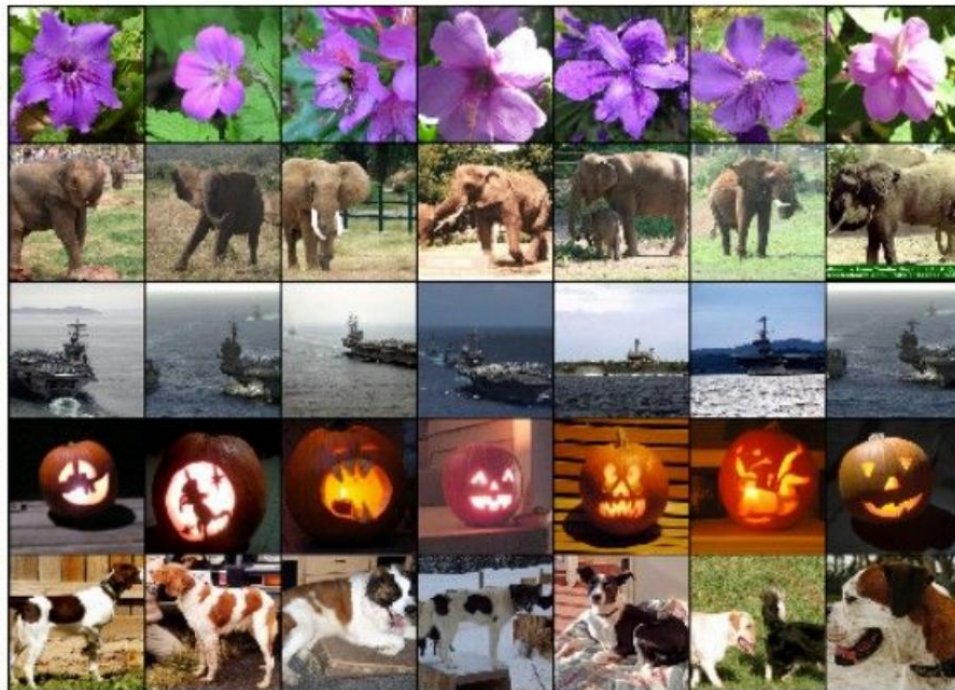
Davi Duarte de Paula

Hoje em dia: ConvNets em todos os lugares

Classification



Retrieval



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.



Photo by Lane McIntosh. Copyright CS231n 2017.

self-driving cars

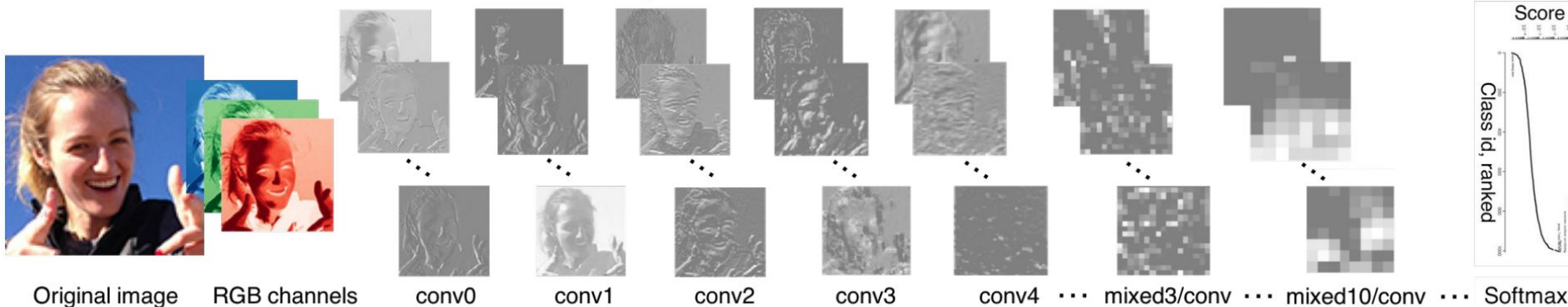


[This image](#) by GBPublic_PR is licensed under [CC-BY 2.0](#)

NVIDIA Tesla line

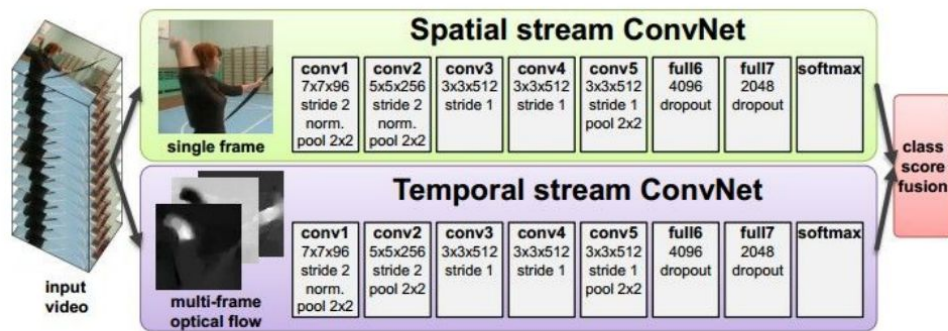
(these are the GPUs on rye01.stanford.edu)

Note that for embedded systems a typical setup would involve NVIDIA Tegras, with integrated GPU and ARM-based CPU cores.



[Taigman et al. 2014]

Activations of [inception-v3 architecture](#) [Szegedy et al. 2015] to image of Emma McIntosh, used with permission. Figure and architecture not from Taigman et al. 2014.



[Simonyan et al. 2014]

Figures copyright Simonyan et al., 2014.
Reproduced with permission.

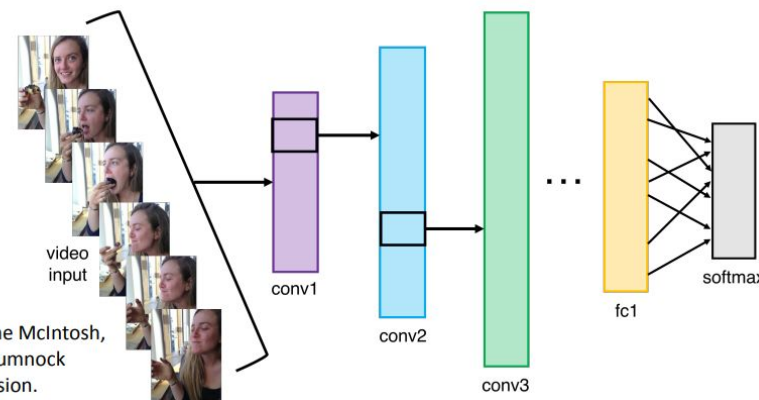
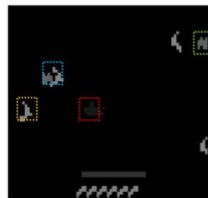
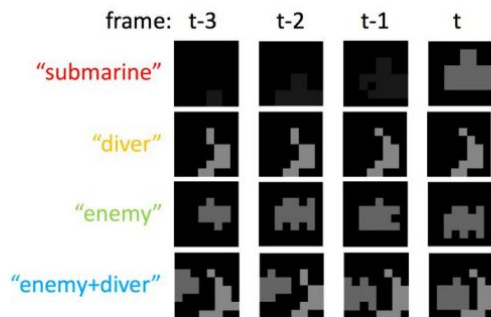


Illustration by Lane McIntosh,
photos of Katie Cumnock
used with permission.



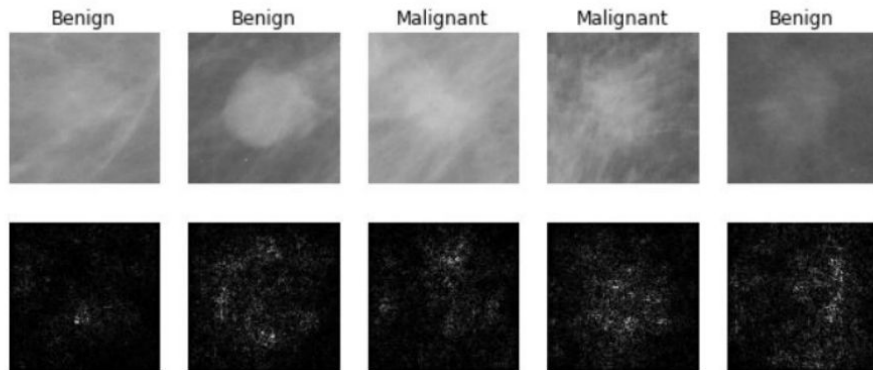
Images are examples of pose estimation, not actually from Toshev & Szegedy 2014. Copyright Lane McIntosh.

[Toshev, Szegedy 2014]



[Guo et al. 2014]

Figures copyright Xiaoxiao Guo, Satinder Singh, Honglak Lee, Richard Lewis, and Xiaoshi Wang, 2014. Reproduced with permission.



[Levy et al. 2016]

Figure copyright Levy et al. 2016.
Reproduced with permission.



[Dieleman et al. 2014]

From left to right: [public domain by NASA](#), usage [permitted](#) by ESA/Hubble, [public domain by NASA](#), and [public domain](#).



[Sermanet et al. 2011]
[Ciresan et al.]

Photos by Lane McIntosh.
Copyright CS231n 2017.

[This image](#) by Christin Khan is in the public domain and originally came from the U.S. NOAA.



Whale recognition, Kaggle Challenge

Photo and figure by Lane McIntosh; not actual example from Mnih and Hinton, 2010 paper.



Mnih and Hinton, 2010

No errors



A white teddy bear sitting in the grass

Minor errors



A man in a baseball uniform throwing a ball

Somewhat related



A woman is holding a cat in her hand

Image Captioning

*[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]*



A man riding a wave on top of a surfboard



A cat sitting on a suitcase on the floor

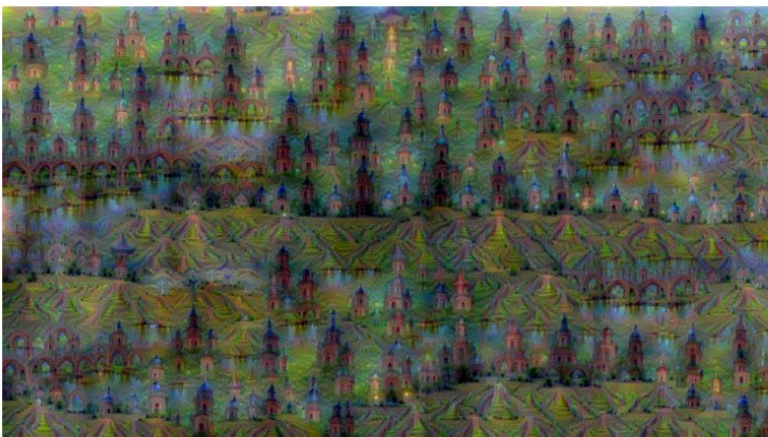
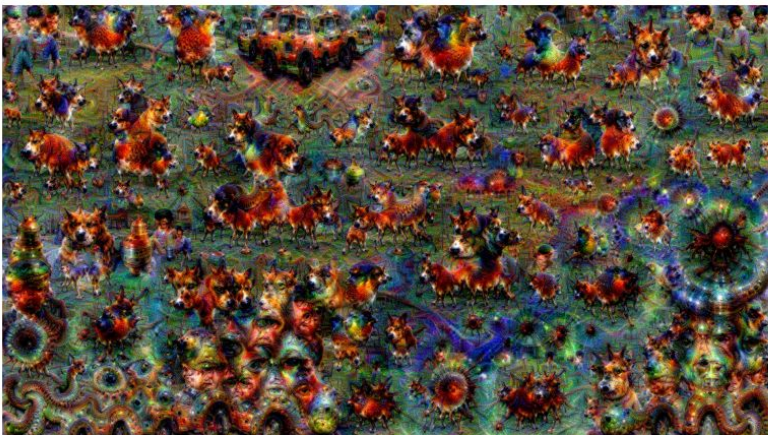


A woman standing on a beach holding a surfboard

All images are CC0 Public domain:

<https://pixabay.com/en/luggage-antique-cat-1643010/>
<https://pixabay.com/en/teddy-plush-bears-cute-teddy-bear-1623436/>
<https://pixabay.com/en/surf-wave-summer-sport-litoral-1668716/>
<https://pixabay.com/en/woman-female-model-portrait-adult-983967/>
<https://pixabay.com/en/handstand-lake-meditation-496008/>
<https://pixabay.com/en/baseball-player-shortstop-infield-1045263/>

Captions generated by Justin Johnson using [NeuralTalk2](#)

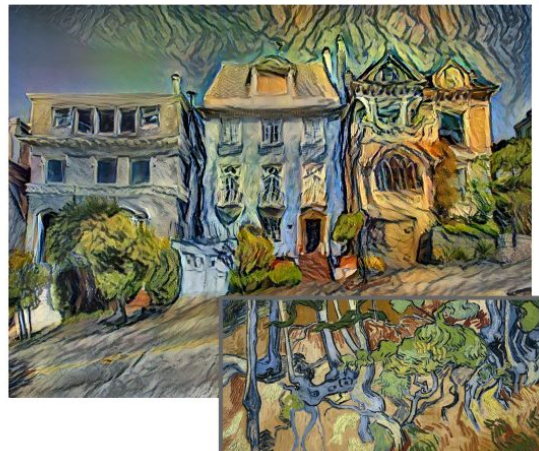


Figures copyright Justin Johnson, 2015. Reproduced with permission. Generated using the Inceptionism approach from a [blog post](#) by Google Research.

Davi Duarte de Paula



Original image is CC0 public domain
[Starry Night](#) and [Tree Roots](#) by Van Gogh are in the public domain
[Bokeh image](#) is in the public domain
 Stylized images copyright Justin Johnson, 2017;

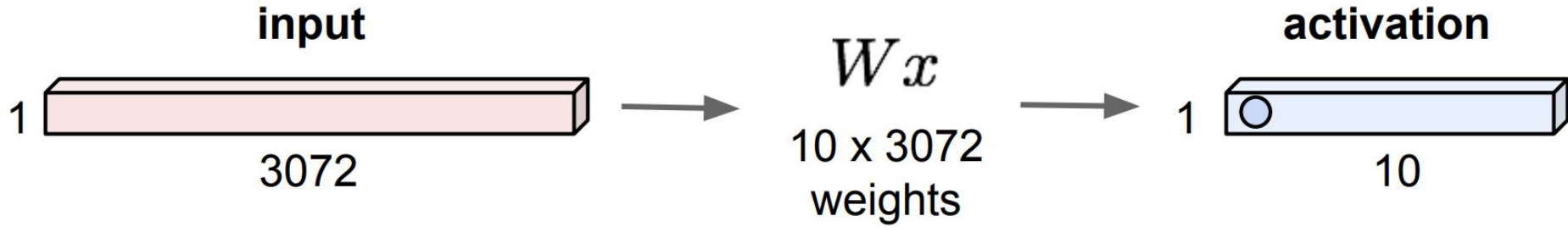


Gatys et al, "Image Style Transfer using Convolutional Neural Networks", CVPR 2016
 Gatys et al, "Controlling Perceptual Factors in Neural Style Transfer", CVPR 2017

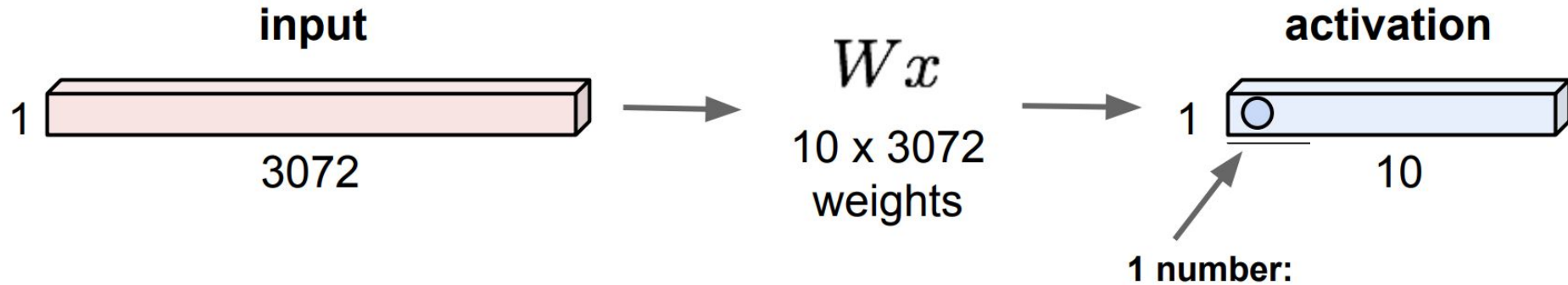
Slide: 23

Convolutional Neural Networks

32x32x3 image -> stretch to 3072 x 1

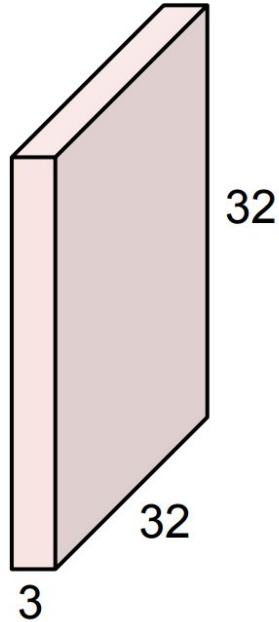


32x32x3 image -> stretch to 3072 x 1



1 número: o resultado de um **produto escalar** entre uma **linha de W** e a **entrada** (um **produto escalar** em 3072 dimensões)

32x32x3 image

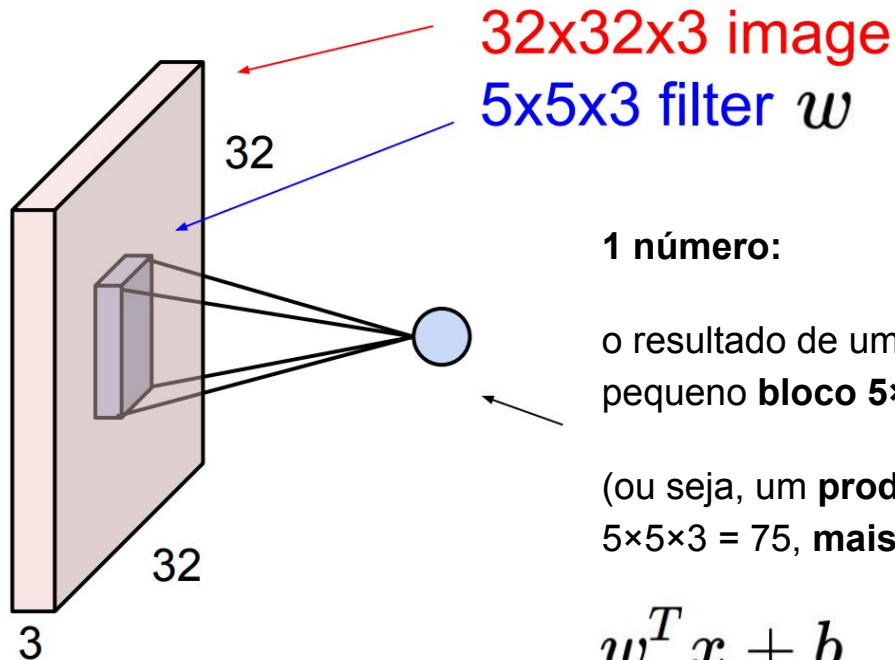


Os filtros sempre se estendem por toda a profundidade do volume de entrada.

5x5x3 filter



Convoluir o filtro com a imagem,
ou seja, "deslizar sobre a imagem
espacialmente, calculando produtos
escalares".

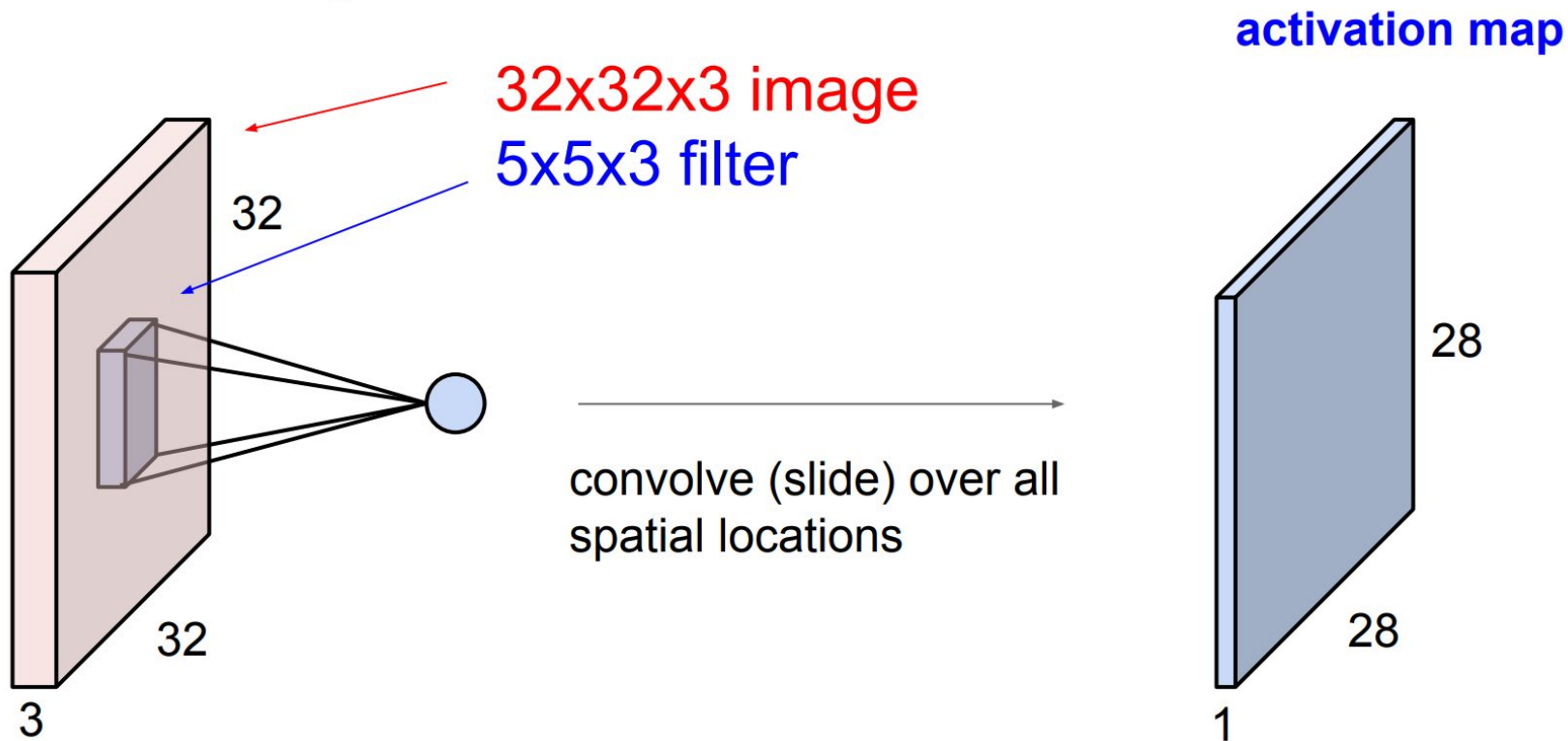


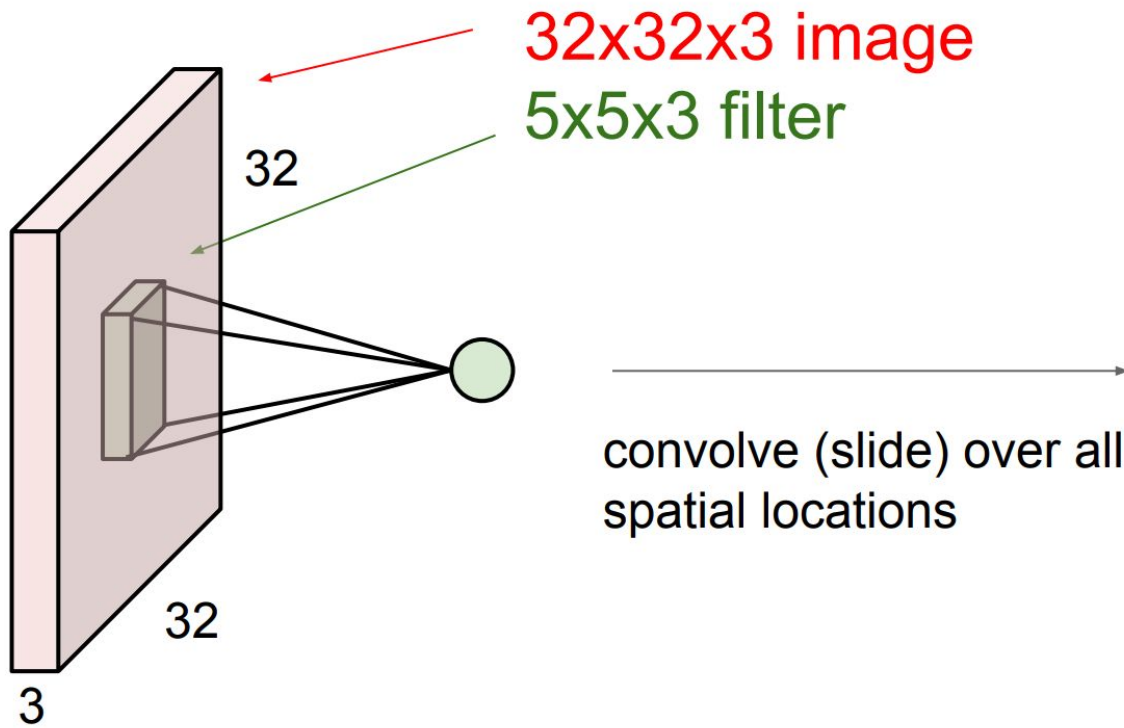
1 número:

o resultado de um **produto escalar** entre o **filtro** e um pequeno **bloco 5x5x3** da imagem

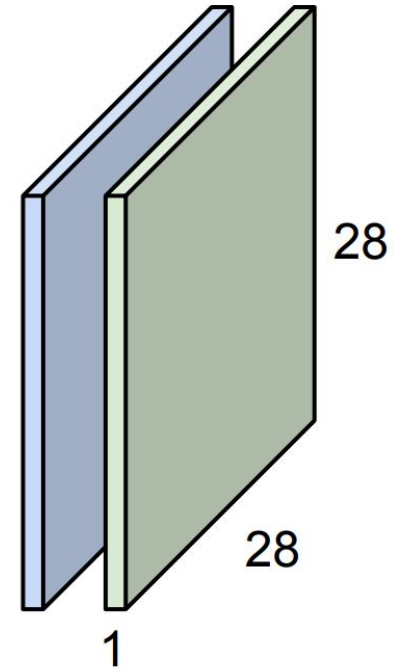
(ou seja, um **produto escalar de 75 dimensões**, já que $5 \times 5 \times 3 = 75$, mais o bias)

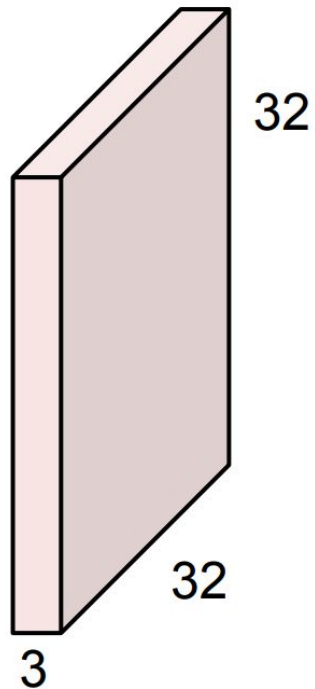
$$w^T x + b$$



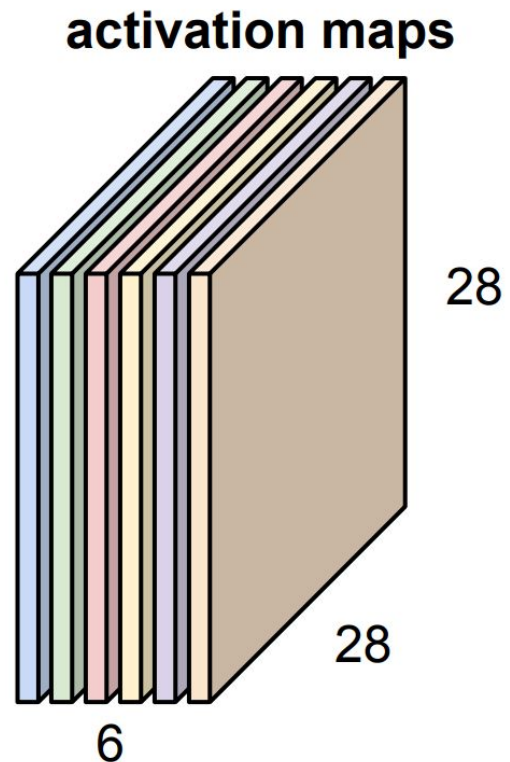


activation maps



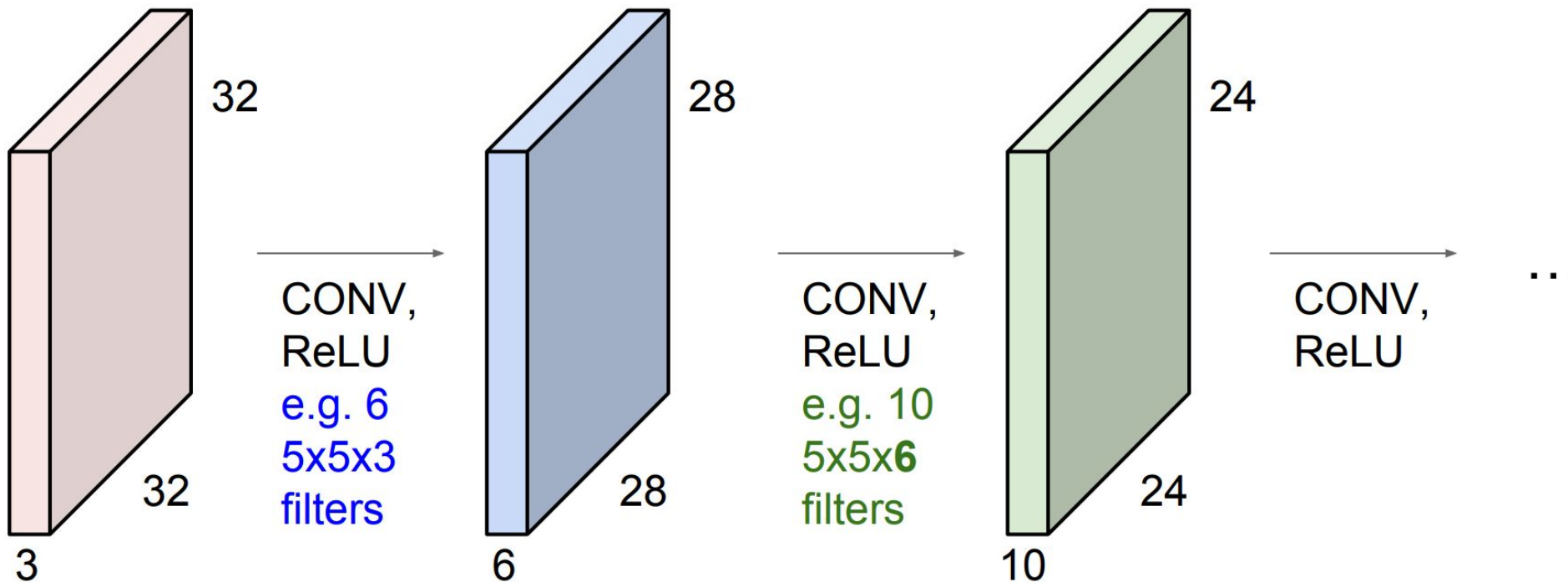


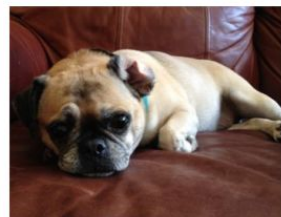
→
Convolution Layer



Empilhamos esses resultados para obter uma “nova imagem” de tamanho $28 \times 28 \times 6$!

Uma **ConvNet** é uma sequência de **camadas convolucionais**, intercaladas com **funções de ativação**.



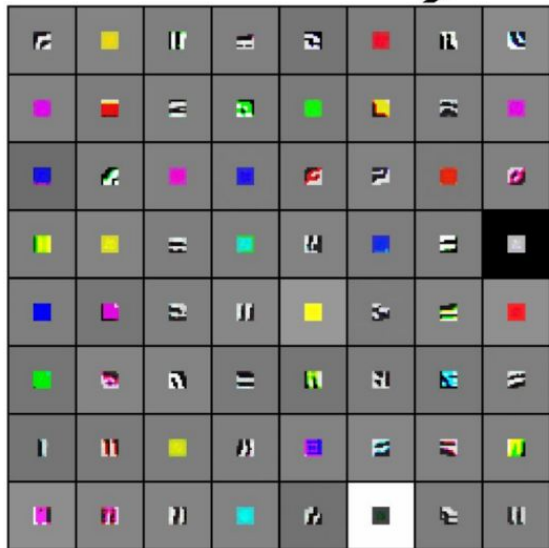


Low-level
features

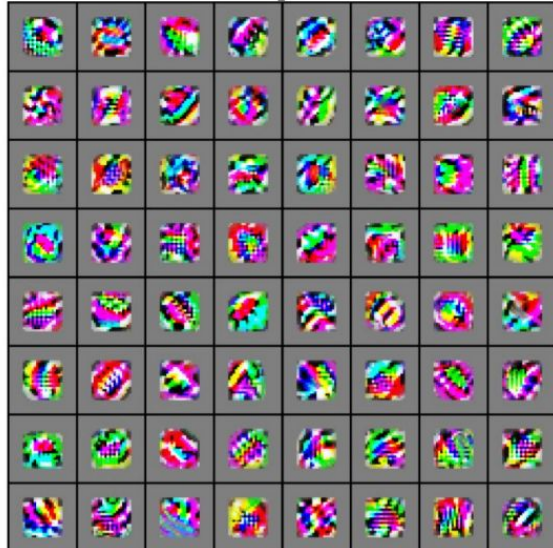
Mid-level
features

High-level
features

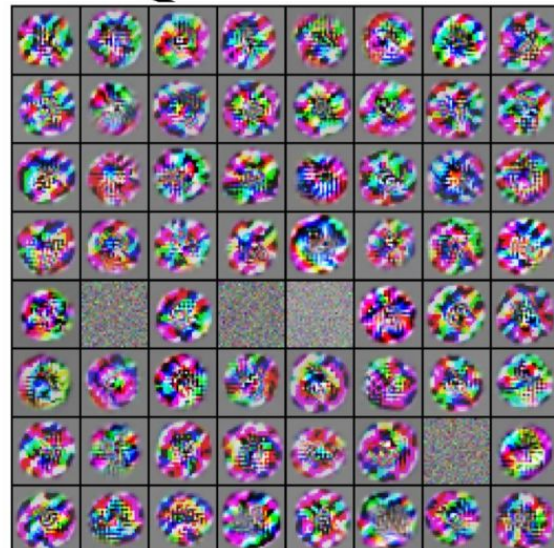
Linearly
separable
classifier



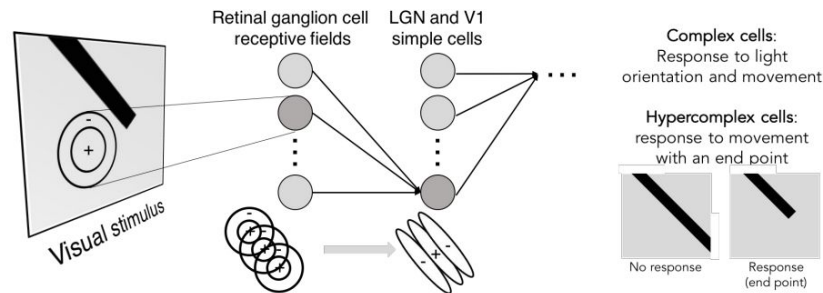
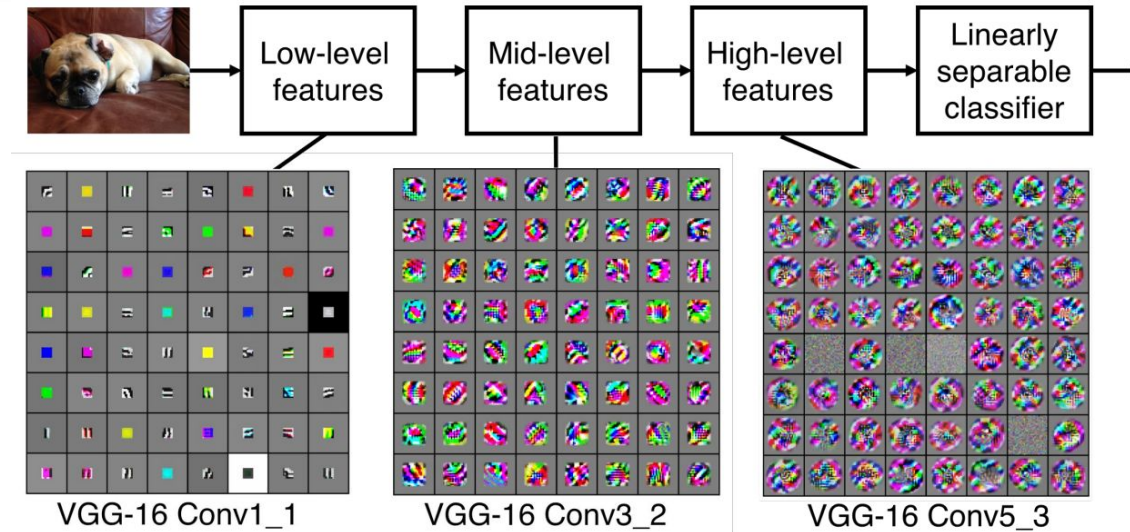
VGG-16 Conv1_1

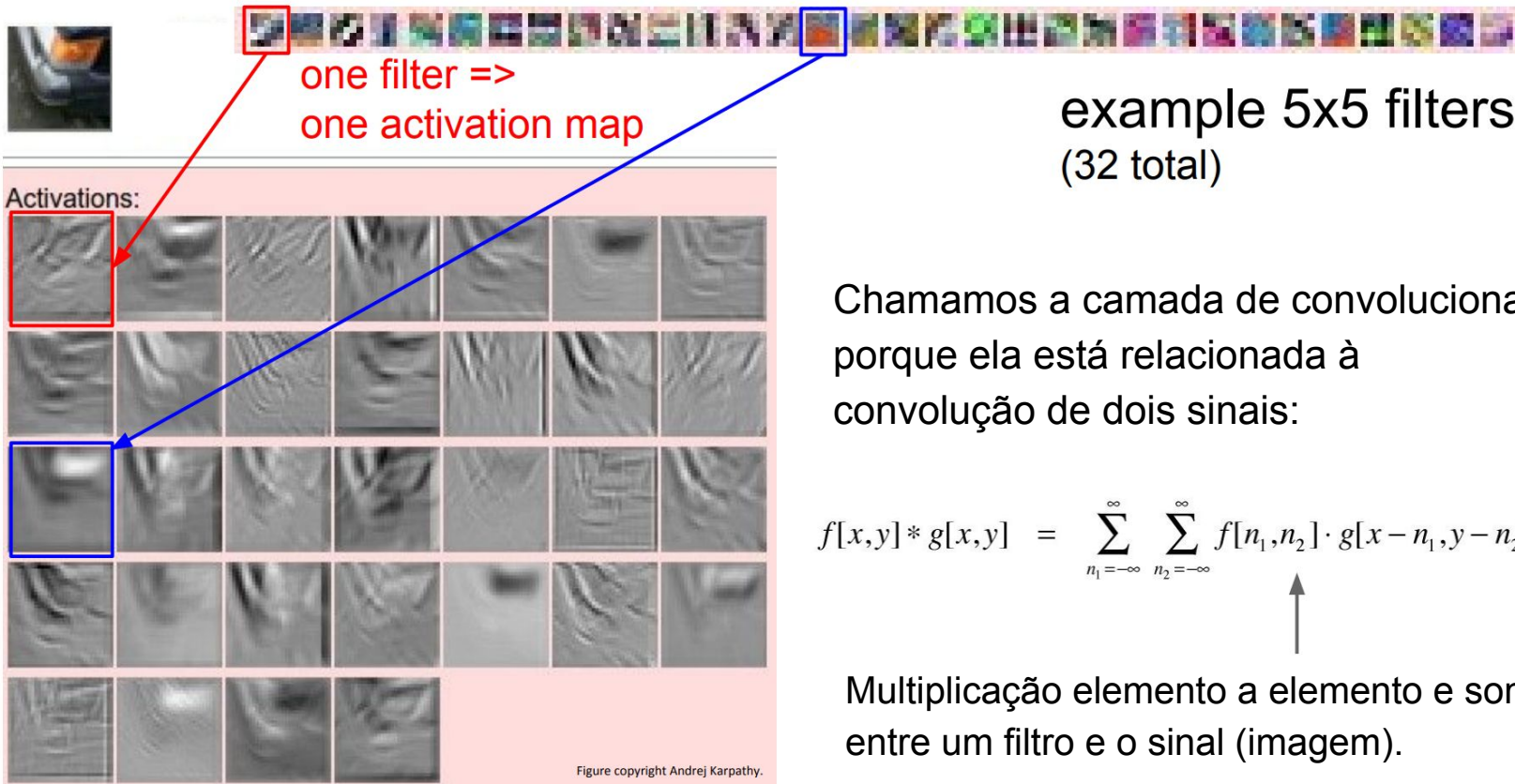


VGG-16 Conv3_2

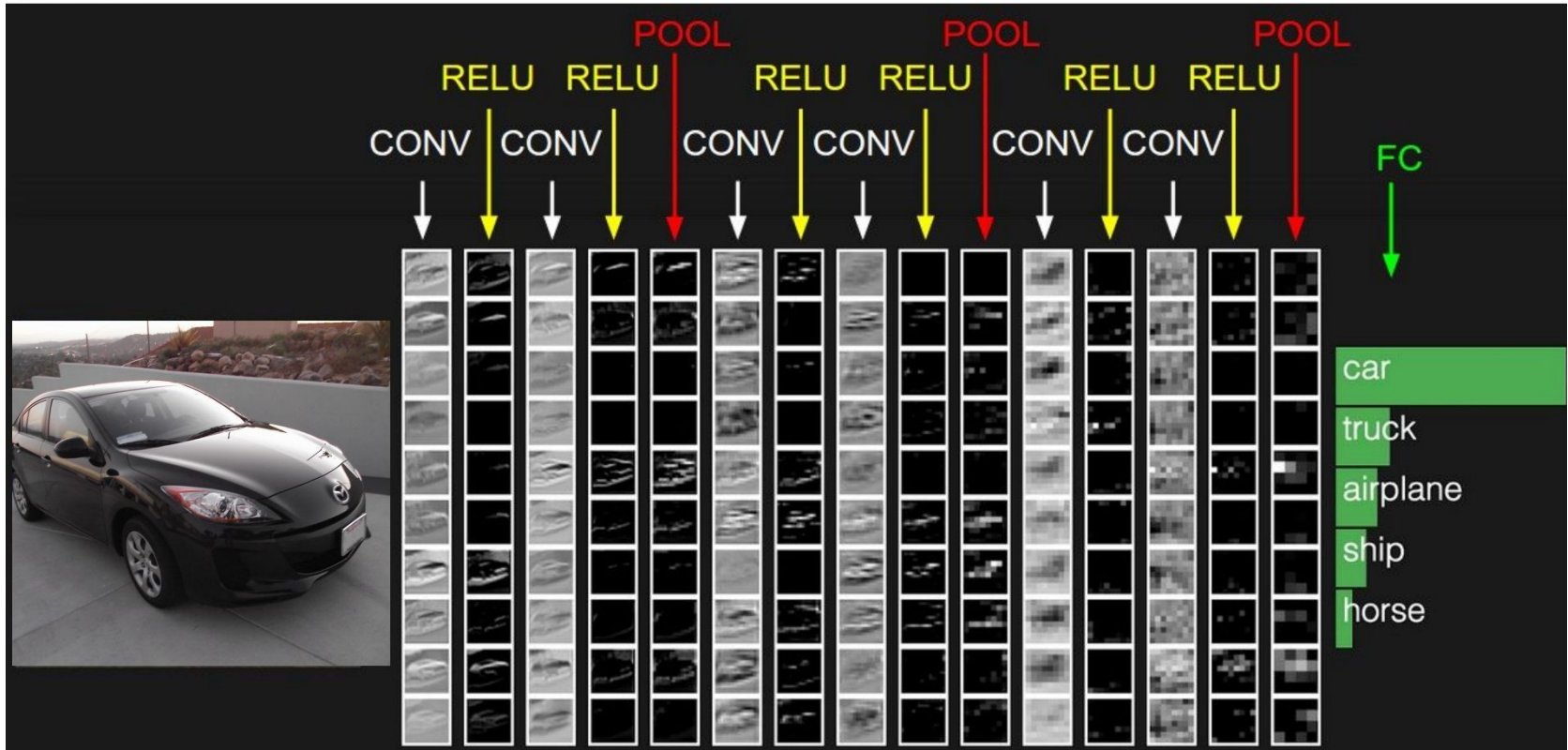


VGG-16 Conv5_3

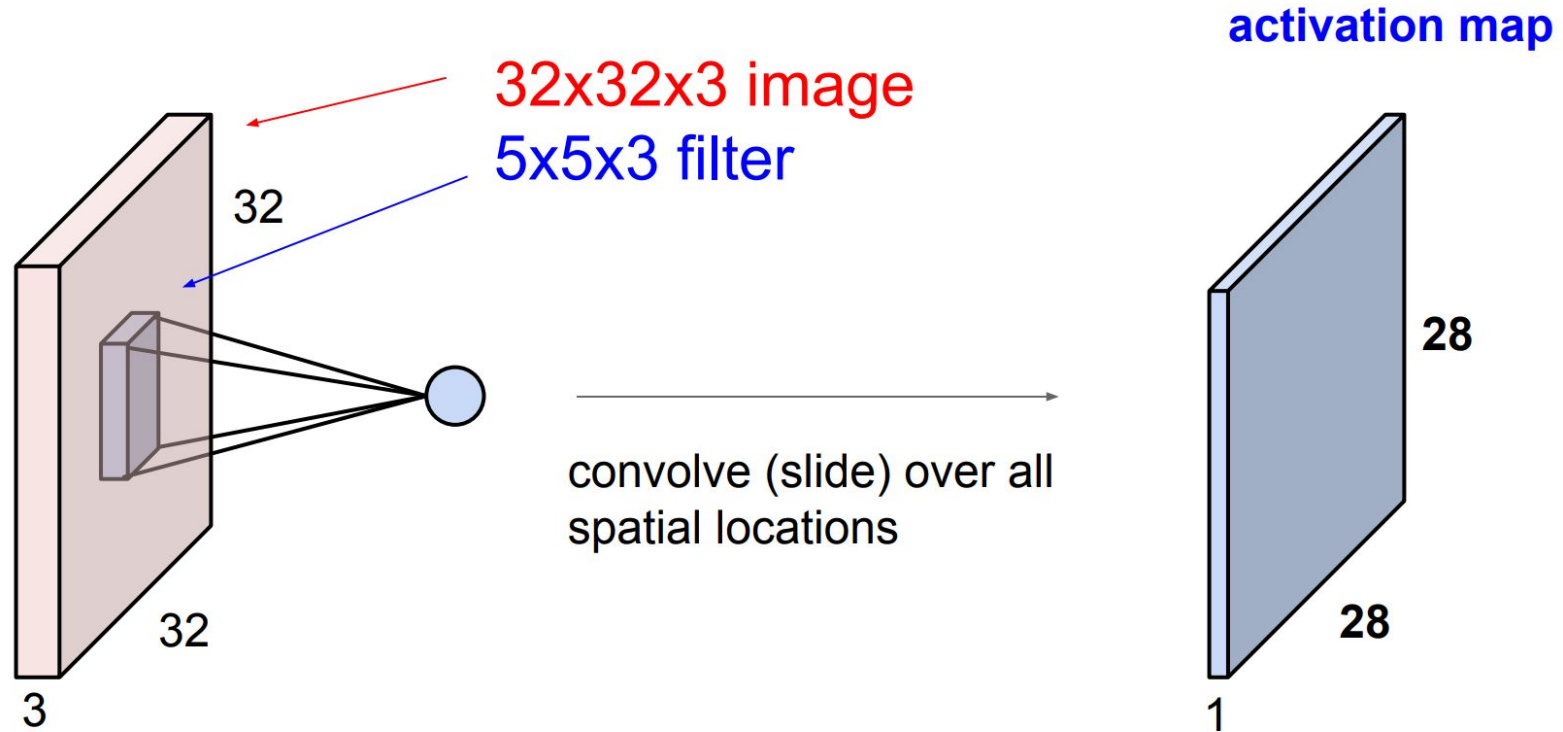




preview:



Atenção especial às dimensões espaciais



7

7x7 entrada (espacial)

assuma filtros 3x3

7

7

7x7 entrada (espacial)

assuma filtros 3x3

7

7

7x7 entrada (espacial)

assuma filtros 3x3

7

7

7x7 entrada (espacial)

assuma filtros 3x3

7

7

7x7 entrada (espacial)

assuma filtros 3x3

7 => Saída de 5x5

7

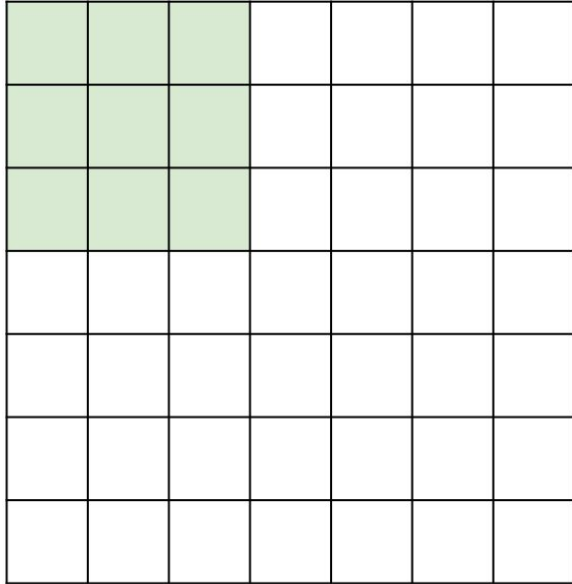
7

7x7 entrada (espacial)

assuma filtros 3x3

aplicados com **stride 2**

7



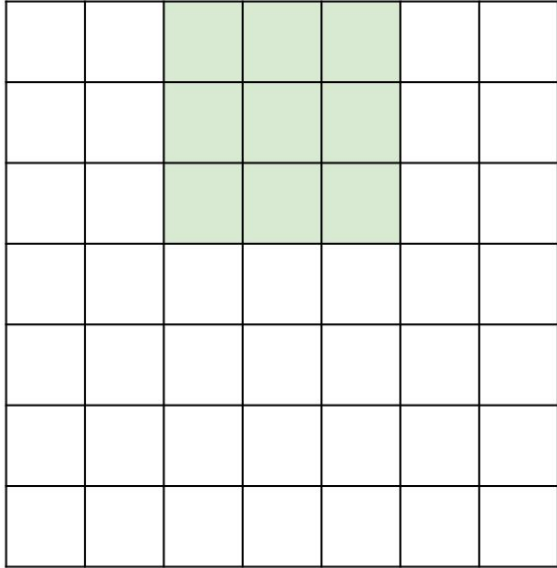
7

7x7 entrada (espacial)

assuma filtros 3x3

aplicados com **stride 2**

7



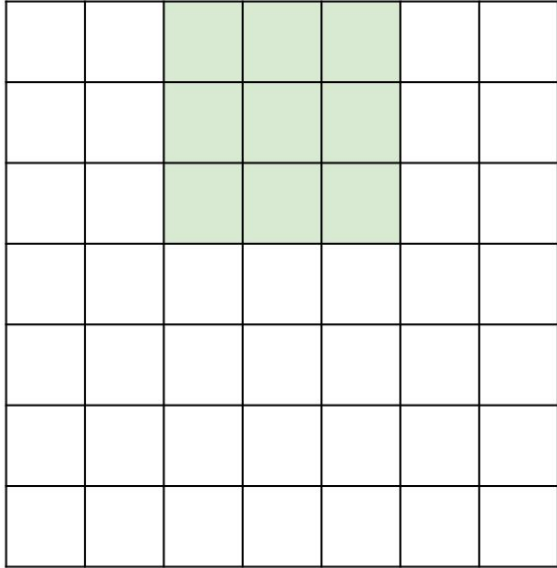
7

7x7 entrada (espacial)

assuma filtros 3x3

aplicados com **stride 2**

7



7

7x7 entrada (espacial)

assuma filtros 3x3

aplicados com **stride 2**

7

7

7x7 entrada (espacial)

assuma filtros 3x3

aplicados com **stride 2**

=> Saída 3x3

7

7

7x7 entrada (espacial)

assuma filtros 3x3

aplicados com **stride 3?**

Não encaixa!

Não é possível aplicar um filtro 3x3 em uma entrada 7x7 com stride 3.

N

N

Tamanho da saída:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7$, $F = 3$:

$$\text{stride } 1 \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{stride } 2 \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3) / 3 + 1 = 2.33 : \backslash$$

0	0	0	0	0	0			
0								
0								
0								
0								

Exemplo:

entrada 7×7

filtro 3×3, aplicado com stride 1

preenchimento (padding) com borda de 1 pixel → qual é a saída?

saída 7×7!

Em geral, é comum ver camadas convolucionais (CONV) com stride 1, filtros de tamanho $F \times F$, e zero-padding com $(F-1)/2$. (isso preserva o tamanho espacial)

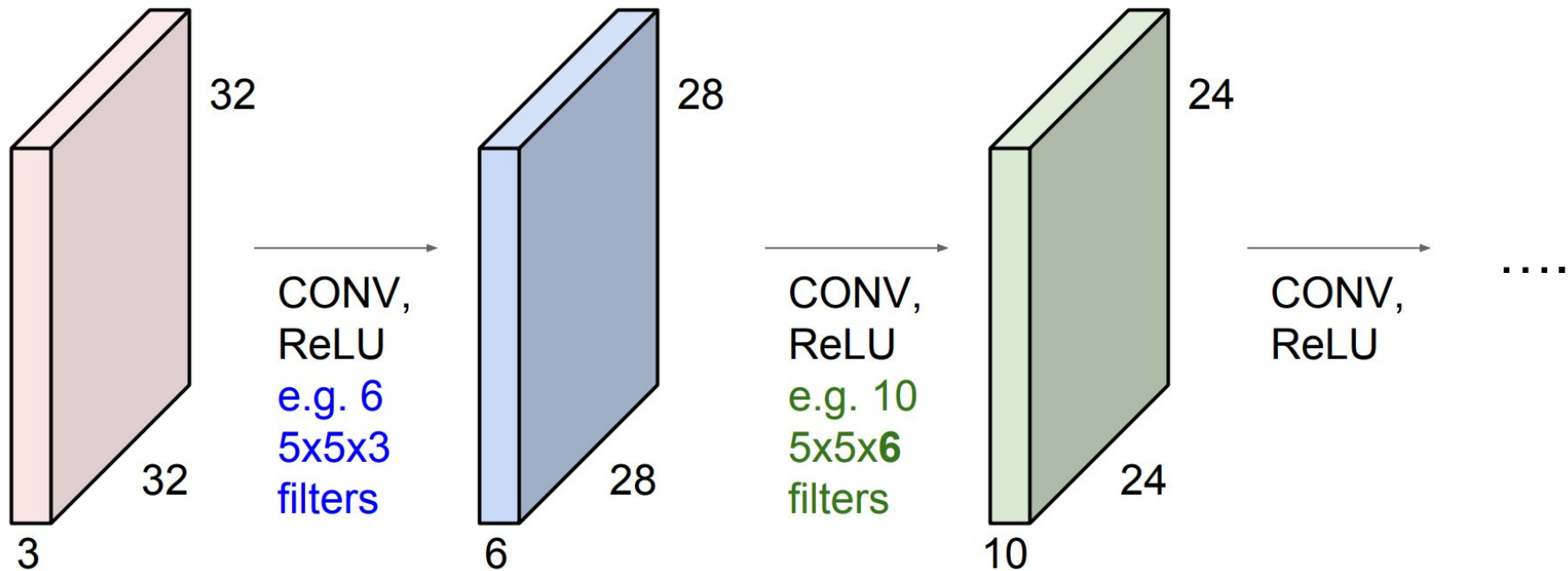
Por exemplo:

$F = 3 \rightarrow$ zero-padding de 1

$F = 5 \rightarrow$ zero-padding de 2

$F = 7 \rightarrow$ zero-padding de 3

Por exemplo, uma entrada 32×32 convoluída repetidamente com filtros 5×5 encolhe o volume espacialmente! ($32 \rightarrow 28 \rightarrow 24 \dots$). Encolher rápido demais não é bom, não funciona bem.



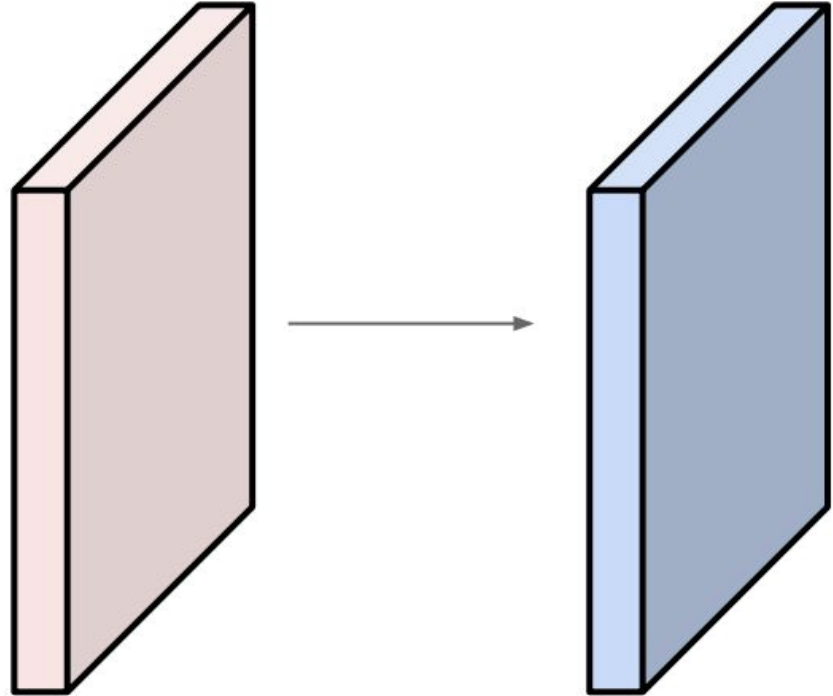
Volume de entrada: $32 \times 32 \times 3$
10 filtros 5×5 , com stride 1 e padding 2

Tamanho do volume de saída:

$$(32 + 2 \times 2 - 5) / 1 + 1 = 32$$

Ou seja, **32 espacialmente**,
então:

$$32 \times 32 \times 10$$



Volume de entrada: $32 \times 32 \times 3$

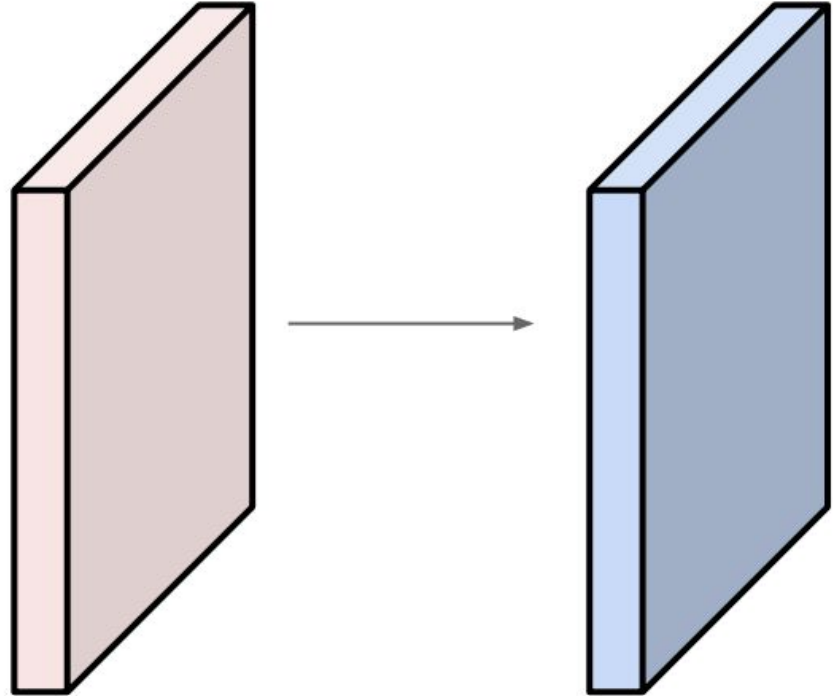
10 filtros 5×5 , com stride 1 e padding 2

Número de parâmetros nesta camada:

Cada filtro tem $5 \times 5 \times 3 + 1 = 76$ parâmetros
(+1 para o bias).

Portanto, o número total de parâmetros
será:

$$76 \times 10 = 760$$



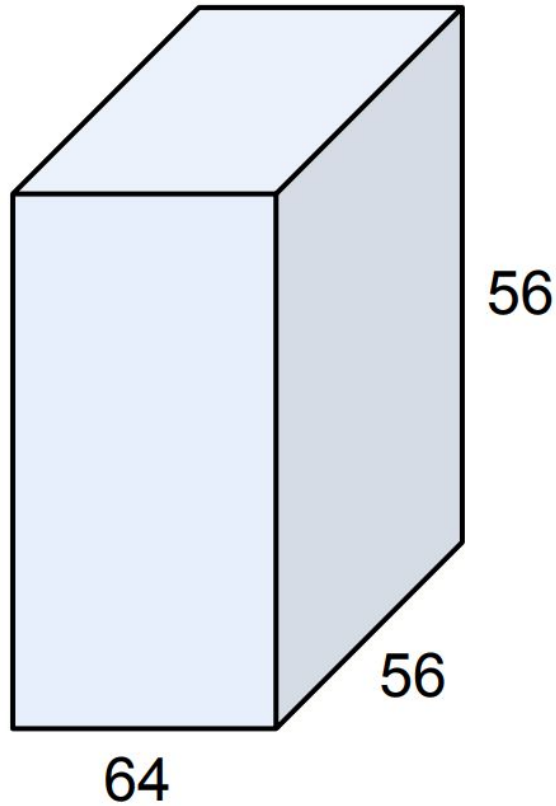
Resumo: Para resumir, a camada convolucional:

- Aceita um volume de tamanho $W_1 \times H_1 \times D_1$
- Requer quatro hiperparâmetros:
 - Número de filtros K
 - Sua extensão espacial F
 - O stride S
 - A quantidade de padding com zeros P
- Produz um volume de tamanho $W_2 \times H_2 \times D_2$, onde:
 - $W_2 = (W_1 - F + 2P)S + 1$
 - $H_2 = (H_1 - F + 2P)S + 1$ (ou seja, a largura e a altura são calculadas de forma simétrica)
 - $D_2 = K$
- Com compartilhamento de parâmetros, introduz $F \times F \times D_1$ pesos por filtro, para um total de $(F \times F \times D_1) \times K$ pesos e K vieses.
- No volume de saída, a fatia de profundidade ddd -ésima (de tamanho $W_2 \times H_2$) é o resultado de realizar uma convolução válida do filtro ddd -ésimo sobre o volume de entrada com stride S , e então ajustada pelo viés ddd -ésimo.

Common settings:

K = (powers of 2, e.g. 32, 64, 128, 512)

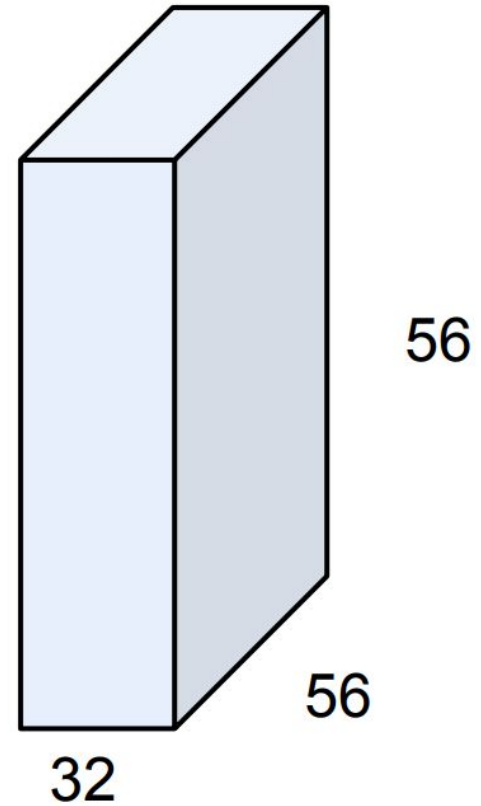
- $F = 3, S = 1, P = 1$
- $F = 5, S = 1, P = 2$
- $F = 5, S = 2, P = ?$ (whatever fits)
- $F = 1, S = 1, P = 0$

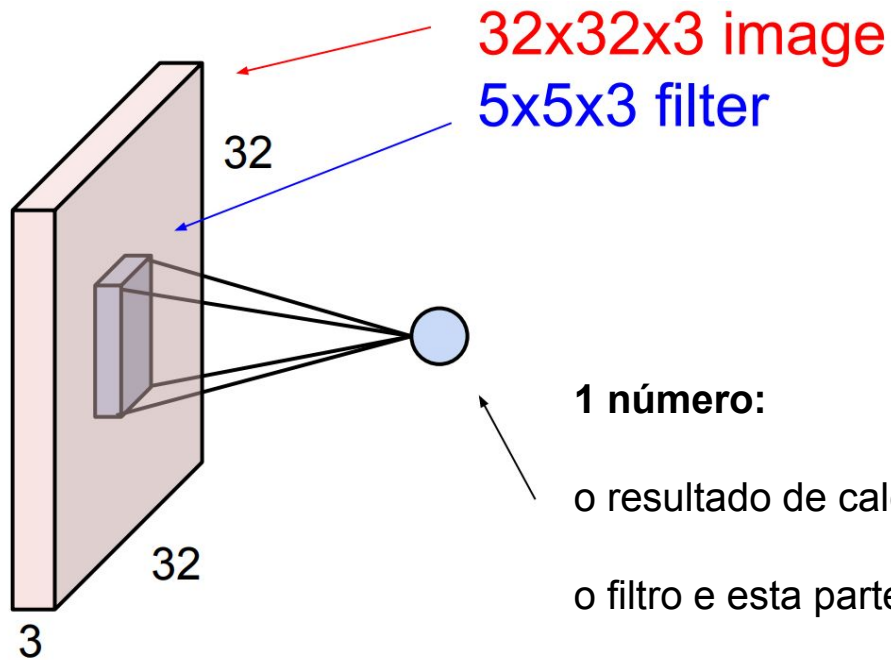


1x1 CONV
with 32 filters



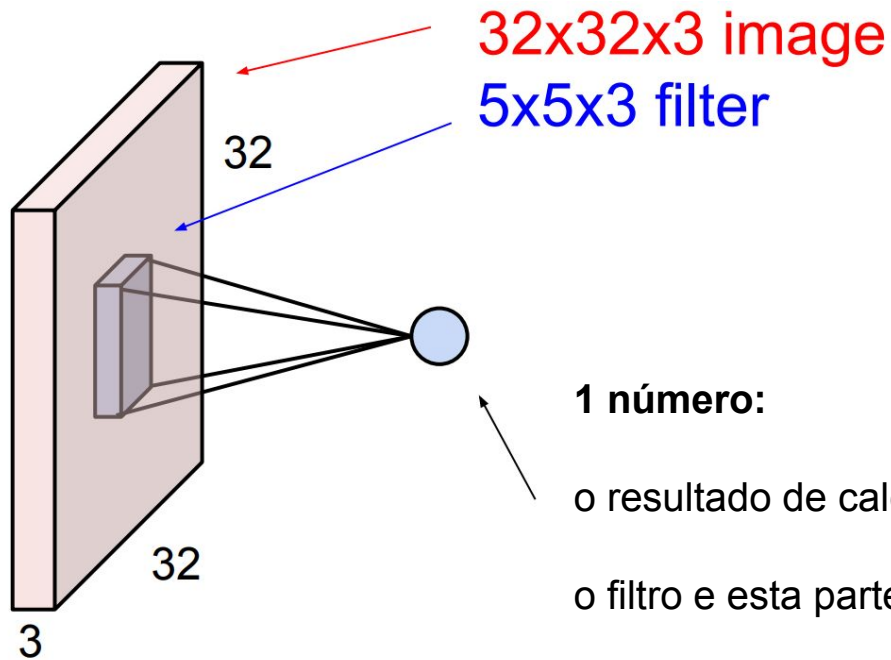
(Cada filtro tem tamanho
1x1x64 e realiza um produto
escalar de 64 dimensões)





o resultado de calcular um produto escalar entre
o filtro e esta parte da imagem

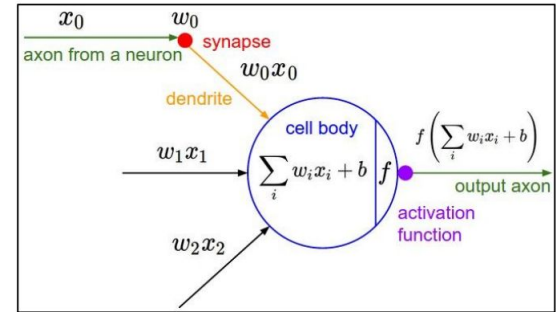
(ou seja, um produto escalar de 75 dimensões, pois
 $5 \times 5 \times 3 = 75$ $\times 5 \times 3 = 75 \times 5 \times 3 = 75$)



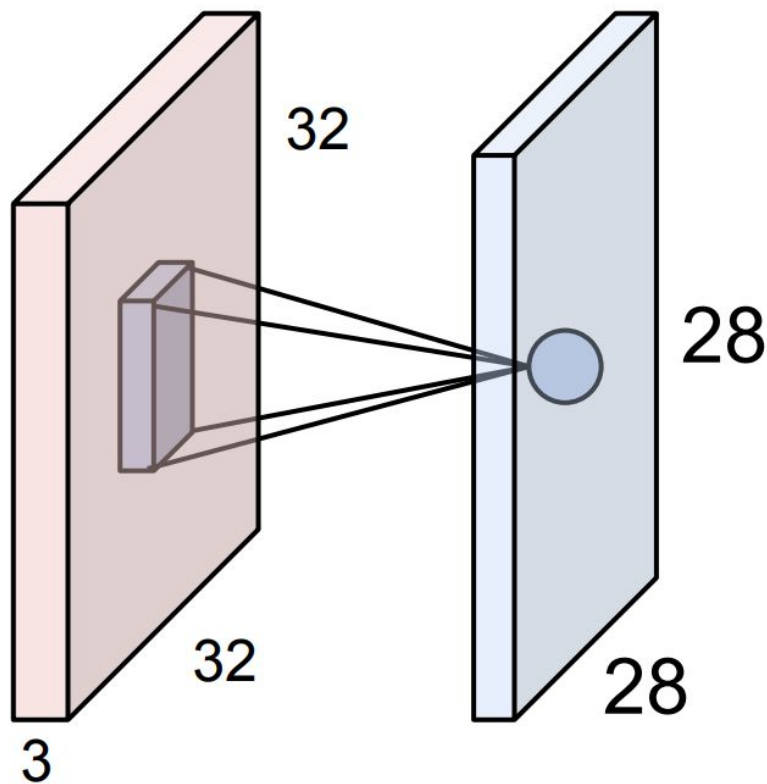
1 número:

o resultado de calcular um produto escalar entre
o filtro e esta parte da imagem

(ou seja, um produto escalar de 75 dimensões, pois
 $5 \times 5 \times 3 = 75$ $\times 5 \times 3 = 75 \times 5 \times 3 = 75$)



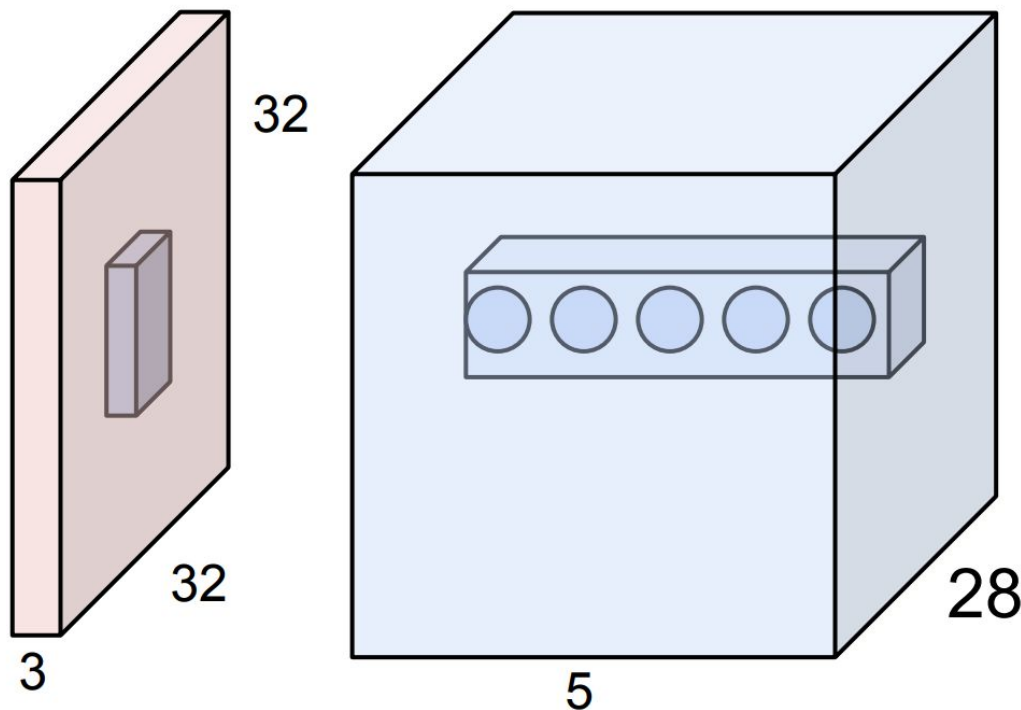
**É APENAS UM NEURÔNIO COM
CONECTIVIDADE LOCAL!**



um **mapa de ativação** é uma folha 28x28 de saídas de neurônios:

1. Cada um está conectado a uma **pequena região** da entrada.
2. Todos compartilham os **mesmos parâmetros**.

28 "**filtros 5x5**" → "**campo receptivo 5x5** para cada neurônio".

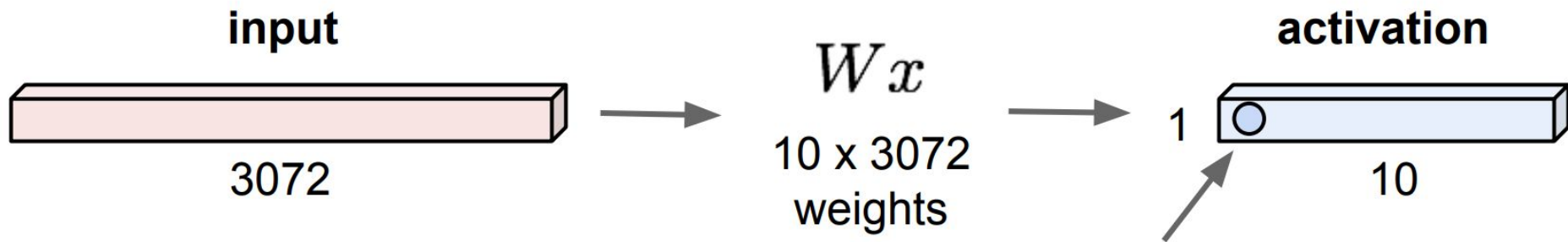


Por exemplo, com **5 filtros**,
a camada **CONV** consiste em
neurônios dispostos em uma
grade 3D
28 (28x28x5).

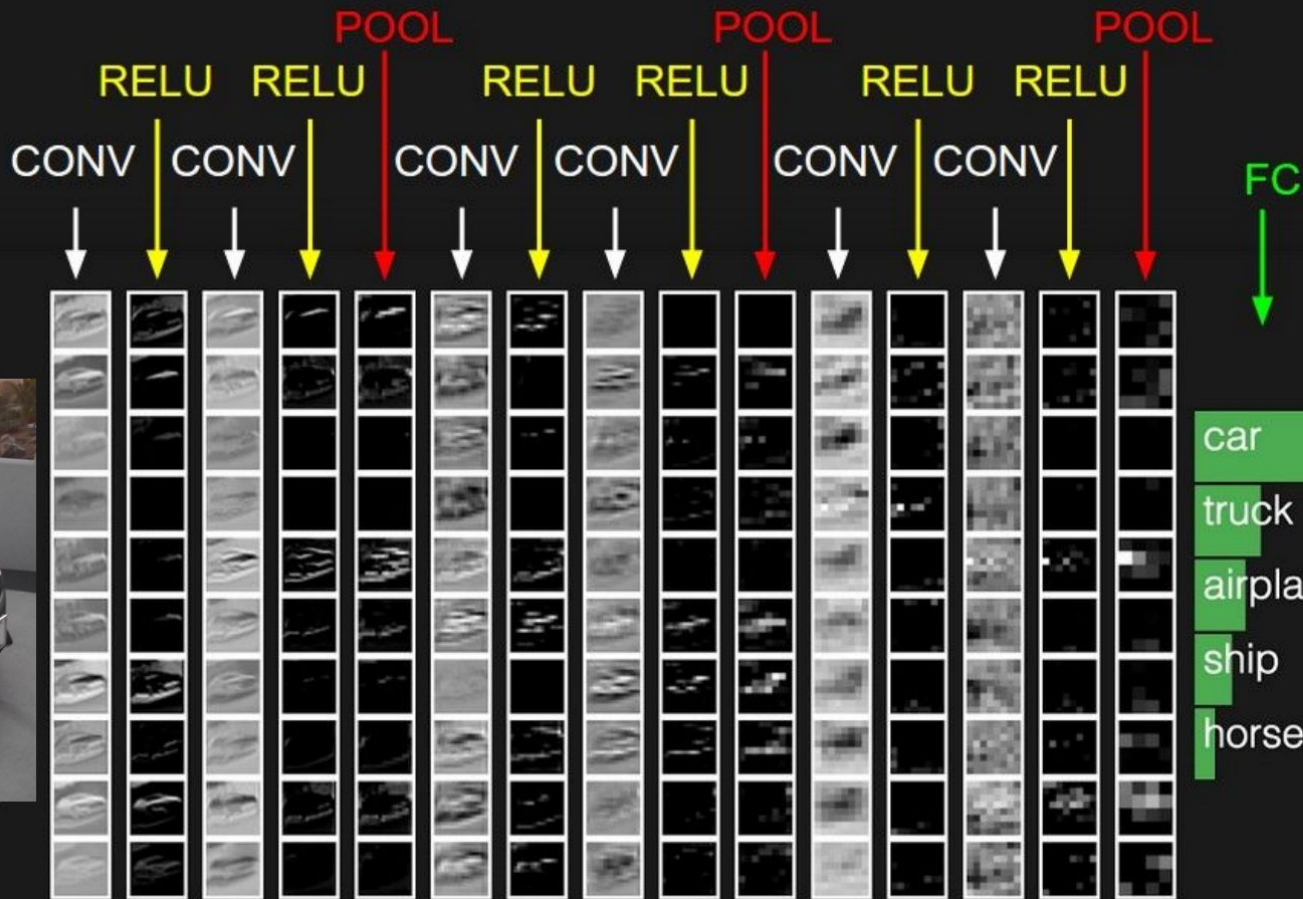
Haverá **5 neurônios diferentes**,
todos observando a mesma
região no **volume de entrada**.

32x32x3 image -> stretch to 3072 x 1

Cada neurônio observa o **volume completo** de entrada.

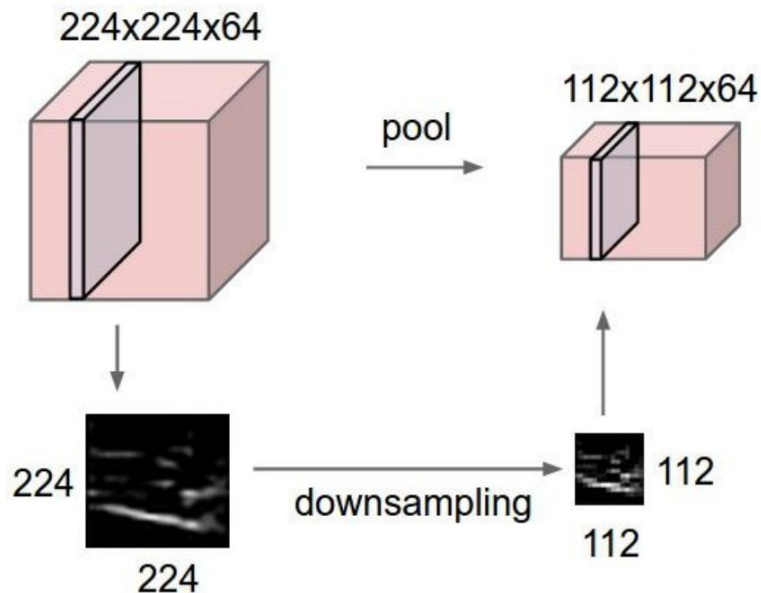


1 número:
o resultado de calcular um **produto escalar** entre uma **linha de W** e a **entrada**
(um **produto escalar de 3072 dimensões**).



Camada de Pooling

- Torna as representações menores e mais manejáveis
- Opera sobre cada mapa de ativação de forma independente:



Single depth slice

x ↑

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

→ y

max pool with 2x2 filters
and stride 2



6	8
3	4

- **Aceita um volume de tamanho** $W_1 \times H_1 \times D_1$
- **Requer três hiperparâmetros:**
 - Sua extensão espacial F ,
 - O stride S ,
- **Produz um volume de tamanho** $W_2 \times H_2 \times D_2$, onde:
 - $W_2 = \frac{(W_1 - F)}{S} + 1$
 - $H_2 = \frac{(H_1 - F)}{S} + 1$
 - $D_2 = D_1$
- **Introduz parâmetros zero**, pois calcula uma função fixa da entrada.
- **Observe que não é comum usar zero-padding para camadas de Pooling.**

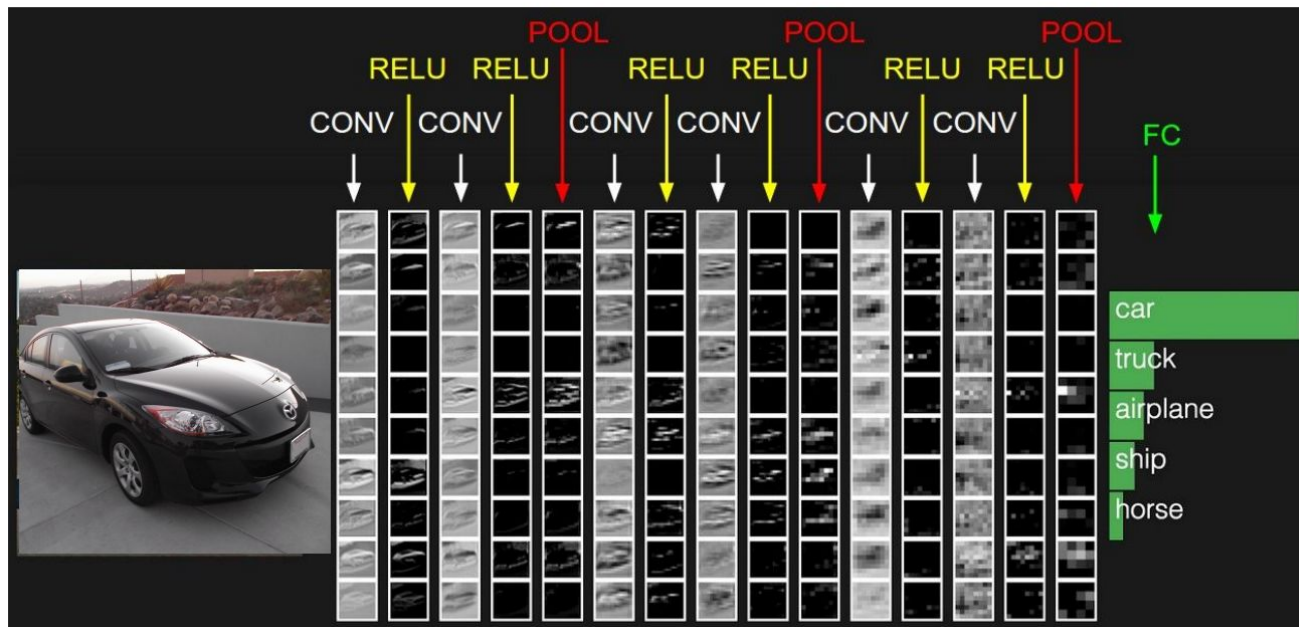
Common settings:

$$F = 2, S = 2$$

$$F = 3, S = 2$$

Camada Completamente Conectada (FC Layer)

Contém neurônios que se conectam a todo o volume de entrada, assim como nas Redes Neurais convencionais.



- Veja código no classroom

Atividade

- Implemente a camada convolucional com e stride. Não utilize funções prontas do Pytorch. Permita configurações de quantidade de filtros, tamanho do filtro, stride, e padding. Utilize alguma imagem de exemplo para mostrar a sua camada funcionando. Mostre como imagem a saída do mapa de ativação para uma imagem e exemplo.